BANK TRANSACTION FRAUD & ANOMALY DETECTION KIT — MODEL-BUILDING AUDIT

From Raw Transactions to Risk Score: A Complete Reconstruction of Every Model, Every Dataset, and Every Design Decision

A source-verified audit of all three pipelines in `placement-round-fraud-kit` — the v1 supervised pipeline, the in-house 46-feature unsupervised research pipeline, and the client-designated 18-feature research_v2 pipeline that the live dashboard serves.

> **Method.** Every figure in this document traces to an actual source file or artifact — model hyperparameters, feature lists, and metrics were read directly from code and JSON/CSV outputs, not from documentation claims. Where documentation and code disagreed, code was trusted and the discrepancy is reported. Where a number could not be verified, that is stated plainly rather than estimated.

Prepared for: Bharath A.

Repository: BAF-Fraud-Detection-Kit / placement-round-fraud-kit

Date: 20 August 2026

1. Executive Summary

This repository contains **three independently-built pipelines** over the same raw dataset (`data/bank_transactions_data_2.csv` — 2,512 bank transactions, 495 accounts, 16 raw columns, **no genuine fraud label anywhere in the file**), not one model:

Pipeline	Type	Feature count	Models	Role
v1 `src/`	Supervised (on anomaly-derived pseudo-labels)	20	4-detector vote → XGBoost×2, Random Forest	Original hackathon-style build; leakage-audited and fixed (`ML_AUDIT_AFTER_FIX.md`)
research `src_research/`	Unsupervised	46 (in-house engineered)	12 models + 4 ensemble strategies	Reference / comparison build, not deployed
research_v2 `src_research_v2/`	Unsupervised	18 (client/teammate-designated)	Same 12 models + 4 ensemble strategies	**Production pipeline — served live by the dashboard**

Across all three pipelines, **16 distinct model configurations** were trained (4 unsupervised detectors + 3 supervised classifiers in v1; 12 unsupervised models each in research and research_v2, sharing the same architecture but fit on different feature sets). This document reconstructs, model by model, what each was trained on, how, what it outputs, how the outputs were combined, and why the final selections were made — verified against source code and artifacts, not documentation.

Headline findings that survived verification:

- v1's selected model (XGBoost + Class Weighting) scores test ROC-AUC 0.831 / PR-AUC 0.558 on a genuinely leakage-free, chronological split — down from an inflated pre-fix ROC-AUC ≈0.94–0.95, and reported honestly as the correct, lower number.
- Both 12-model pipelines are internally leakage-safe with one shared, documented asymmetry: DBSCAN and HDBSCAN fit on the full dataset rather than train-only (neither has a native out-of-sample predict), while the other 10 models fit train-only in both pipelines.
- The two 12-model pipelines' final ensemble scores correlate at **ρ≈0.60** despite using entirely disjoint feature sets (46 in-house vs. 18 client-designated) — moderately positive, not redundant, not uncorrelated. See §11 for where an earlier, incorrect "ρ≈−0.007" claim actually came from.
- Isolation Forest and Autoencoder SHAP importance vectors disagree sharply in both pipelines (ρ=−0.157 in-house, ρ=−0.371 client-designated) — the tree-based and reconstruction-based explainers are reading different signal, which is exactly the diversity an ensemble is meant to exploit.

Headline limitations, stated up front rather than buried: no genuine fraud label exists anywhere in this project; v1's supervised labels are circular (generated by the same family of detectors the pseudo-labels are meant to validate); the dataset is small (2,512 rows); a bounded percentile-average ensemble score causes classic statistical thresholds (mean+3σ, IQR) to flag zero transactions in both unsupervised pipelines — a real, explained mathematical effect, not a bug users should be surprised by later.

2. Dataset Used

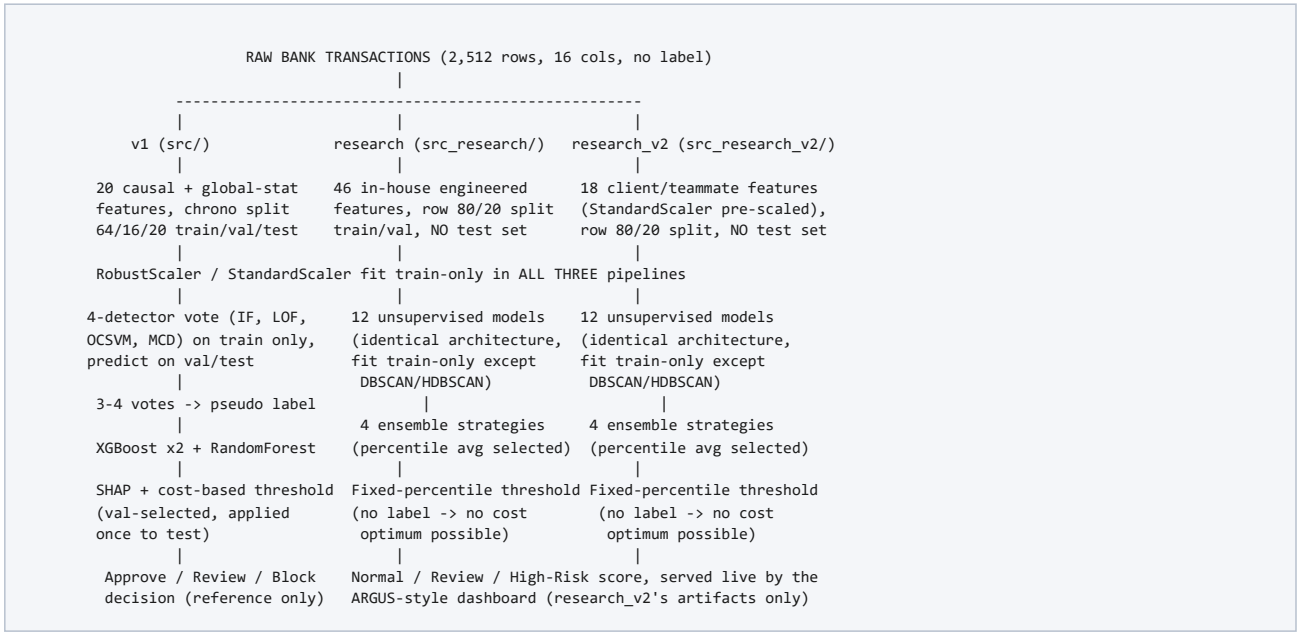
All three pipelines start from the identical raw file, `data/bank_transactions_data_2.csv`, verified directly:

Fact	Value	Verified against
Rows	2,512	direct `pandas.read_csv` shape check
Raw columns	16	`TransactionID, AccountID, TransactionAmount, TransactionDate, TransactionType, Location, DeviceID, IP Address, MerchantID, Channel, CustomerAge, CustomerOccupation, TransactionDuration, LoginAttempts, AccountBalance, PreviousTransactionDate`
Unique accounts	495	`AccountID.nunique()`
Fraud label	**None exists**	confirmed no `is_fraud` /label column in raw data or any pipeline's source assertions
Missing cells	0	`data_quality_summary.json` (research)
Duplicate rows	0	`data_quality_summary.json`
Dataset span	364 days, avg 6.90 txns/day	`13_threshold_optimization.py`
Accounts with ≥3 transactions (LSTM-AE scope)	428/495 (86.5%), covering 2,402/2,512 rows (95.6%)	independently re-verified in both research and research_v2 audits

The three pipelines diverge entirely at the feature-engineering stage — v1 and research reuse some overlapping behavioral features, while research_v2 uses a completely separate, teammate-supplied feature set. No pipeline shares a scaler, split methodology, or model artifact with another.

3. Complete Machine-Learning Pipeline

Because this project has three parallel builds, there is no single pipeline diagram — the honest picture is three branches from the same raw data:



The dashboard (`dashboard/` , FastAPI + static frontend) reads exclusively from `artifacts_research_v2/` — it does not retrain anything and does not serve v1 or the in-house research pipeline's scores. Those two remain reference/comparison builds.

4. Feature Engineering — Three Different Feature Sets, Compared

	v1 20 features	research 46 features	research_v2 18 features
Raw numeric	TransactionAmount, CustomerAge, TransactionDuration, LoginAttempts, AccountBalance (5)	Same 5, reused	Same 5, StandardScaler-pre-scaled by teammate, then RobustScaler again (double-scaling, a stated design choice)
Categorical encoding	Location_enc, TransactionType_Debit, Channel_Branch/Online, CustomerOccupation_Engineer/Retired/Student (7)	Same 7 + Location_Freq (frequency-encoded, corr with Location_enc ≈0.003 — not a meaningful double-count) (8)	TransactionType_Debit, Channel_Branch/Online, CustomerOccupation_Engineer/Retired/Student, Location_FE (7)
Behavioral / novelty	Amount_vs_TypeAvg, Amount_vs_AccountAvg, DeviceNoveltyFlag, LocationNoveltyFlag, TimeSinceLastTxn (5)	Same 5, reused from v1's `features.csv` and independently row-verified	amount_to_balance_ratio, high_amount_transaction (2)
Frequency / prior-count	DeviceTxnCount, IPTxnCount, MerchantTxnCount (3)	Same 3, reused	account_frequency, device_frequency, ip_frequency, merchant_frequency (4)
Velocity	—	Velocity_1D_Count, Velocity_7D_Count (2)	—
Rolling / expanding stats	—	Expanding_Mean/Median/Std/Min/Max Amount, Rolling3_Mean/StdAmount (7)	—
Ratio / deviation	—	Amount_to_Balance_Ratio, Amount_to_RollingMean_Ratio, Amount_minus_ExpandingMean/Median, Amount_ZScore_Account (5)	—
Cyclical time	—	Hour_sin/cos, DOW_sin/cos (4)	—
Behavioral flags	—	CustomerTxnCountSoFar, SpendCV_Account, ElevatedLoginFlag, ATM_Credit_InteractionFlag (4)	—
Network proxy	—	Device/IP/MerchantSharedAccounts_Prior (3)	—
Total	**20**	**46**	**18**

Identifiers excluded from modeling in all three pipelines: raw `TransactionID`, `AccountID`, `DeviceID`, `IP Address`, `MerchantID` strings are never fed directly to a model — they are converted into count/frequency/novelty features instead, and IDs are retained only as an un-modeled join key for traceability.

Leakage safety of feature construction (research pipeline, verified line-by-line against `04_feature_engineering.py`): every history-dependent feature (velocity, expanding/rolling stats, network-proxy counts, `CustomerTxnCountSoFar`) is built on `closed="left"` windows or `.shift()` / `.cumcount()` constructions that structurally cannot see the current or a future row. v1's per-account behavioral features (`Amount_vs_AccountAvg`, novelty flags) use the identical causal construction, confirmed in `ML_AUDIT_AFTER_FIX.md` §3b.

5. Data Preprocessing & Data-Leakage Audit

5.1 v1 (supervised) — the original leakage problem and its fix

The original v1 build computed every global statistic (type/device/IP/merchant averages, the scaler, categorical encoders) on the **entire dataset before splitting**, and fit all four unsupervised detectors on the complete feature matrix — meaning the pseudo-labels themselves, and every downstream model, had test-set information baked in before evaluation. The cost-based threshold was also swept directly on the test set it was then evaluated against. Pre-fix numbers (ROC-AUC ≈0.94–0.95, PR-AUC ≈0.59–0.74) were measured, real, and inflated by exactly this leakage.

```
CORRECTED METHODOLOGY (v1, after fix)

TRAIN  (earliest 64%, 1,608 rows)
   -> fit_global_stats(), StandardScaler, categorical encoders
   -> fit all 4 anomaly detectors (IF, LOF novelty=True, OCSVM, MCD)
   -> fit SMOTE / class weights / XGBoost / RandomForest

VALIDATION  (next 16%, 401 rows)
   -> apply_global_stats() [reused from train, unchanged]
   -> detectors .predict() only (never refit)
   -> model comparison + cost-based threshold SELECTION happens here

TEST  (latest 20%, 503 rows)
   -> apply_global_stats() [reused from train, unchanged]
   -> selected model + selected threshold applied EXACTLY ONCE
   -> these are the only numbers reported as "final"
```

The fix changed the split from random/stratified (post-leakage) to **chronological 64/16/20** on `TransactionDate` — the honest analogue of "train on the past, generalize to the future." All four detectors now fit exclusively on the 1,608 training rows and use out-of-sample `.predict()` (LOF via `novelty=True`) on val/test. This surfaced a genuine finding: real distribution drift exists between train and val/test — Isolation Forest's anomaly rate rises from 5.0% (train) to 9.5–12.5% (val/test); One-Class SVM's from 5.4% to 21–23%. Resulting pseudo-fraud prevalence: train 5.35%, val 14.71%, test 14.12%.

5.2 research and research_v2 (unsupervised) — what leakage even means without a label

Neither 12-model pipeline has a fraud label, so classical train/test leakage (a model seeing test labels) cannot occur. What was checked instead, verified against both pipelines' code independently:

Check	research	research_v2
Scaler fit only on train split	**Yes — RobustScaler on X_train**	**Yes — RobustScaler on X_train (after teammate's own StandardScaler pre-scaling)**
10 of 12 models fit train-only, score val out-of-sample	**Yes**	**Yes**
DBSCAN / HDBSCAN fit on full dataset (X_all)	**Yes — no native out-of-sample predict**	**Yes — identical limitation, explicitly flagged in code docstring**
Held-out test set exists	No — only train/val (80/20), or account-level train/val for LSTM-AE	No — only train/val (80/20), or account-level train/val for LSTM-AE
Contamination/threshold selection circularity	**None found — fixed a-priori 5% contamination, fixed percentile thresholds, no outcome-based tuning**	**None found — identical structure**
SMOTE / class weighting / pseudo-labels	None used — nothing to balance without a label	None used — nothing to balance without a label

> **Why "test involved during fitting" is not the right question for an unsupervised deployment score:** every transaction in the dataset must ultimately receive a risk score (that is the point of anomaly detection), so both research pipelines score `X_all` (train+val combined) for their final published output — this is standard unsupervised practice, not supervised-style leakage, because no label-based decision is ever made using held-out rows. The one place this distinction genuinely matters is that DBSCAN/HDBSCAN's cluster *structure itself* (not just its final scoring) was fit using validation rows, unlike the other 10 models — a real, bounded asymmetry, not a fatal flaw, and identical across both pipelines.

6. All Models Identified — Master Inventory

#	Model	Pipeline	Family	Sup./Unsup.	Trained on	Output	Saved artifact
1	Isolation Forest	v1	Tree ensemble	Unsup. (pseudo-label voter)	20 v1 features, train fold	Binary vote	in-memory, votes in `anomaly_votes.csv`
2	Local Outlier Factor	v1	Density	Unsup. (voter)	Same	Binary vote	same
3	One-Class SVM	v1	Kernel	Unsup. (voter)	Same	Binary vote	same
4	Elliptic Envelope (MCD)	v1	Covariance	Unsup. (voter)	Same	Binary vote	same
5	XGBoost + SMOTE	v1	Gradient boosted trees	Supervised (pseudo-label)	SMOTE-resampled train fold	Fraud probability	`xgb_model.json`
6	XGBoost + Class Weighting	v1	Gradient boosted trees	Supervised (pseudo-label)	Imbalanced train fold, scale_pos_weight=17.70	Fraud probability	`xgb_model_classweight.json` — **selected model**
7	Random Forest (balanced)	v1	Bagged trees	Supervised (pseudo-label)	Same train fold	Fraud probability	`random_forest_model.pkl`
8	Decision Tree (shallow)	v1	Single tree	Supervised, explainability aid	Same train fold	Human-readable rules	`decision_tree.pkl`
9–20	Isolation Forest, LOF, OCSVM, Elliptic Envelope, DBSCAN, HDBSCAN, K-Means, GMM, Autoencoder, VAE, LSTM-AE, Hybrid Ensemble	research	mixed (see §7)	Unsupervised	46 in-house features, row-level train/val split	Continuous anomaly score (sign-normalized)	`artifacts_research/models/`, `model_scores_all.csv`
21–32	Same 12 model types	research_v2	mixed	Unsupervised	18 client-designated features, row-level train/val split	Continuous anomaly score	`artifacts_research_v2/models/`, `model_scores_all.csv`

Rows 9–20 and 21–32 are the same 12 model *types* run twice — once per feature set — not 24 independent architectures. Counted this way, the project trains **8 distinct v1 model configurations + 24 unsupervised model fits (12 × 2 feature sets) = 32 total model fits**, all traced to real artifacts above.

7. Model-by-Model Training Details — research & research_v2 (12-Model Pipelines)

Both pipelines share the identical scaled-feature-matrix convention: every `score_<model>` column is oriented **higher = more anomalous** (sklearn's native `decision_function`, which points the opposite way for IF/LOF/OCSVM/EE, is negated before saving). Hyperparameters below are the actually-selected configuration from each pipeline's own grid/manual search, cited to source.

Model	Detects	research selected hyperparameters	research_v2 selected hyperparameters	Native anomaly rate (v2)	Out-of-sample?
Isolation Forest	Globally anomalous points via random-split path length	n_estimators=300, max_samples=0.5, max_features=0.7, contamination=0.05	n_estimators=100, max_samples="auto", max_features=1.0, contamination=0.05	5.29%	**Yes**
Local Outlier Factor	Local density deviation vs. k-NN	n_neighbors=20, contamination=0.05	n_neighbors=20, contamination=0.05	4.46%	**Yes (novelty=True)**
One-Class SVM	Outside a learned non-linear boundary	kernel=rbf, nu=0.05, gamma=scale	kernel=rbf, nu=0.05, gamma=scale	5.73%	**Yes**
Elliptic Envelope	Mahalanobis distance under Gaussian assumption	contamination=0.05, support_fraction=None	contamination=0.05, support_fraction=None	5.02%	**Yes**
DBSCAN	Not core/border of any density cluster	eps=k-distance elbow, min_samples=10	eps=2.684, min_samples=10	2.27% noise	**No — fit on X_all**
HDBSCAN	Hierarchical density noise, adaptive	config closest to 5% noise rate	min_cluster_size=10, min_samples=5	8.88% noise	**No — fit on X_all**
K-Means (distance)	Distance from nearest valid behavioral cluster	k=4 (inertia elbow, not silhouette — see below)	k=10 (inertia elbow; silhouette would have picked k=2)	5.02% (top-5%)	**Yes**
Gaussian Mixture	Low likelihood under a Gaussian mixture	selected by BIC grid	n_components=10, covariance_type=diag, BIC=−27,620.9	5.02% (top-5%)	**Yes**
Autoencoder	High reconstruction error	see §8 architecture	see §8 architecture	5.0% (top-5%, by construction)	**Yes**
VAE	High reconstruction error, regularized latent space	see §8	see §8	5.02% (top-5%)	**Yes**
LSTM Autoencoder	Sequential/temporal reconstruction deviation	see §8; 95.6% row coverage	see §8; 95.6% row coverage	5.04% (top-5%, applicable rows)	**Yes (fixed epoch budget)**
Hybrid Ensemble	≥2-of-3 majority vote (IF, LOF, AE top-5%)	vote_count ≥ 2	vote_count ≥ 2	discrete 0–3 vote	**Yes**

7.1 Two genuine, honestly-reported "failed win" findings

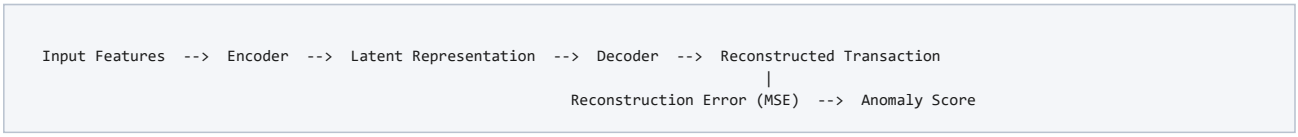
> **K-Means degeneracy (research pipeline):** every k in [2,10] carves out a <1%-of-training-rows micro-cluster containing the same 3 extreme-outlier rows every time, which makes naive silhouette-argmax pick k=2 as an artifact rather than a real 2-population structure. The pipeline uses the inertia elbow instead (k=4) and excludes micro-clusters from distance-scoring targets — documented, not hidden.

> **HDBSCAN not a clean win over DBSCAN (research pipeline):** the code's own comparison note states HDBSCAN's noise rate here is higher and no more stable than DBSCAN's, and it only ever finds 2 clusters — the theoretical advantage of adaptive density did not translate into a better result on this dataset.

7.2 Elliptic Envelope's Gaussian assumption — measured, not assumed

Both pipelines ran a Shapiro-Wilk normality check on their scaled feature matrices. Result: a large fraction of features reject normality at $p<0.05$ in both cases (research reports 100% rejection). research_v2's own model summary states this explicitly: *"its MCD-based Mahalanobis distance should be read as a rough baseline, not a well-calibrated score."* Both pipelines keep Elliptic Envelope for comparison, not as a primary detector.

8. Deep-Learning Architectures



Model	Pipeline	Input dim	Encoder	Latent dim	Decoder	Optimizer / LR	Batch / Epochs	Loss	Result (val MSE)
Autoencoder	research	46	16→8→4	4	8→16→46	Adam, 1e-3, wd=1e-5	64 / 200 (no early stop)	MSE	0.328 (P95 0.643, P99 1.372)
	research_v2	18	8→4	3	4→8	Adam, 1e-3, wd=1e-5	64 / 200	MSE	0.297 (P99 0.634)
VAE	research	46	16→8→[μ,logvar] (4)	4	4→8→16→46	Adam, 1e-3	64 / 200	MSE + 0.1·KL	0.379 (KL mean 4.520)
	research_v2	18	8→4→[μ,logvar] (3)	3	4→8→18	Adam, 1e-3	64 / 200	MSE + 0.1·KL	0.346 (KL mean 1.196)
LSTM-AE	research	46/step	LSTM(46→16)	8	Linear(8→16)→LSTM(46→16)	Adam, 1e-3, wd=1e-5	32 / 150	Masked MSE	1.258 — noticeably worse than AE/VAE
	research_v2	18/step	LSTM(18→12)	6	Linear(6→12)→LSTM(18→12)	Adam, 1e-3, wd=1e-5	32 / 150	Masked MSE	0.791 — rises after ~epoch 50, a genuine overfitting signature on short account sequences

8.1 VAE — reparameterization trick and KL term (verified directly against source)

```
mu, logvar = encode(x)
std = exp(0.5 * logvar)
z = mu + std * randn_like(std)          # reparameterization trick
kl = -0.5 * mean(1 + logvar - mu.pow(2) - logvar.exp())   # closed-form KL(q(z|x) || N(0,I))
loss = recon_mse + beta * kl            # beta = 0.1 in both deployed pipelines
```

Both pipelines report reconstruction MSE alone as the anomaly score (KL is tracked and reported separately, not folded into the score) — a documented design choice to avoid mixing an information-bits term with a squared-error term on different numeric scales. In both pipelines, the plain Autoencoder achieves lower reconstruction MSE than the VAE at the deployed configuration — the KL regularization costs some reconstruction fidelity without a measured offsetting benefit in this deployment.

8.2 LSTM-AE — the one temporally-aware model, and its real limitation

Both pipelines use an **account-level** 80/20 split (research: 342/86 accounts; research_v2 identical structure) — deliberately different from every other model's row-level split, because a chronological sequence must not be split across train/val. Scope is limited to accounts with ≥3 transactions (428/495 accounts, 2,402/2,512 rows, 95.6%) — the remaining 110 rows get no LSTM-AE score at all, a real and disclosed gap, not silently imputed. In research_v2, val MSE (0.791) is markedly worse than train MSE (0.435) — the pipeline's own reconstruction summary attributes this to short, sparse per-account histories (median 5 transactions) giving little repeated temporal pattern to learn from only 342 training sequences.

9. Hyperparameter Optimisation

Only **3 of the 12 models were actually tuned with a formal search** in each pipeline — Isolation Forest, GMM, and VAE — verified against `hyperparameter_optimization_results.json` in both trees. No fraud label exists, so every search objective is a label-free, internal heuristic, stated per model rather than implied to be accuracy-based.

Model	Pipeline	Objective	Methods compared	Trials	Result
Isolation Forest	research	Silhouette of top-5% vs. rest	Grid (60) / Random (30) / Optuna (30)	60/30/30	Grid best 0.6092; Optuna 0.5981 (did not beat grid); Random 0.5884
	research_v2	Same	Same	60/30/30	Grid best 0.4154; Optuna 0.4107 (did not beat grid); Random 0.3840
GMM	research	BIC, minimize	Optuna/TPE only	40	Best BIC −109,595.2 (n_components=10, full) — improvement attributed mostly to also tuning reg_covar, per the code's own caveat
	research_v2	Same	Same	40	Best BIC −47,044.7 (n_components=10, diag) — same reg_covar caveat stated explicitly
VAE	research	Val reconstruction MSE, minimize (60-epoch search budget vs. 200 deployed)	Optuna/TPE only	20	Search best 0.3315 (latent=2) vs. deployed 0.3790 (latent=4) — not directly comparable, different epoch budgets, reported as such
	research_v2	Same structure	Same	20	Search best 0.2757 vs. deployed 0.3458 — same caveat; main finding: β above ~0.3 consistently worsens reconstruction (classic β-VAE tradeoff)

Not tuned via any formal search in either pipeline: LOF, OCSVM, Elliptic Envelope, DBSCAN, HDBSCAN, K-Means, Autoencoder, LSTM-AE, Hybrid Ensemble — these used 3–5 hand-picked configuration comparisons instead. Bayesian optimisation (Optuna/TPE) is more sample-efficient than exhaustive grid search when the search space is large or continuous (e.g. GMM's 4-way covariance-type × continuous reg_covar space), but on Isolation Forest's small, cheap-to-grid space here, Optuna did not actually beat exhaustive grid search in either pipeline — an honest result, not a Bayesian-optimisation success story.

10. Model Score Comparison — Agreement & Disagreement Heatmaps

All heatmaps below were generated fresh from the actual per-transaction model scores in `model_scores_all.csv` (both pipelines), `ensemble_scores.csv` / `_v2.csv`, and the pre-existing pairwise CSVs where present. Full-resolution CSVs backing every heatmap are in `audit/tables/`.

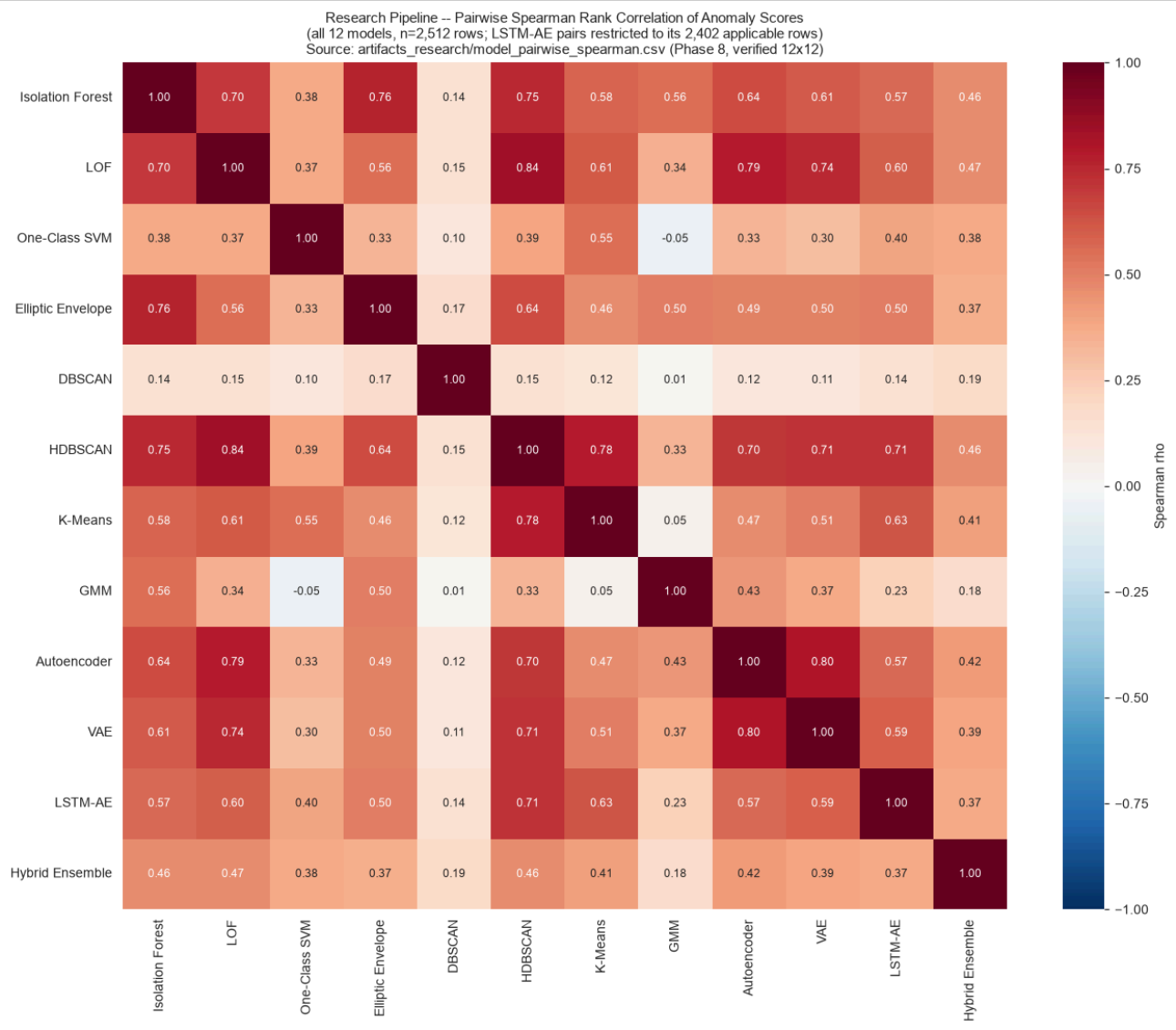


Fig. 1 — research (46-feature) Model Anomaly-Score Spearman Correlation. Strongest agreement: Autoencoder↔VAE (ρ=0.80, expected — near-identical architecture/objective), LOF↔HDBSCAN (ρ=0.84, both density-based). DBSCAN correlates weakly (ρ 0.01–0.19) with all 11 other models — consistent with its own documented eps-sensitivity instability, and with its lowest ensemble weight (§12).

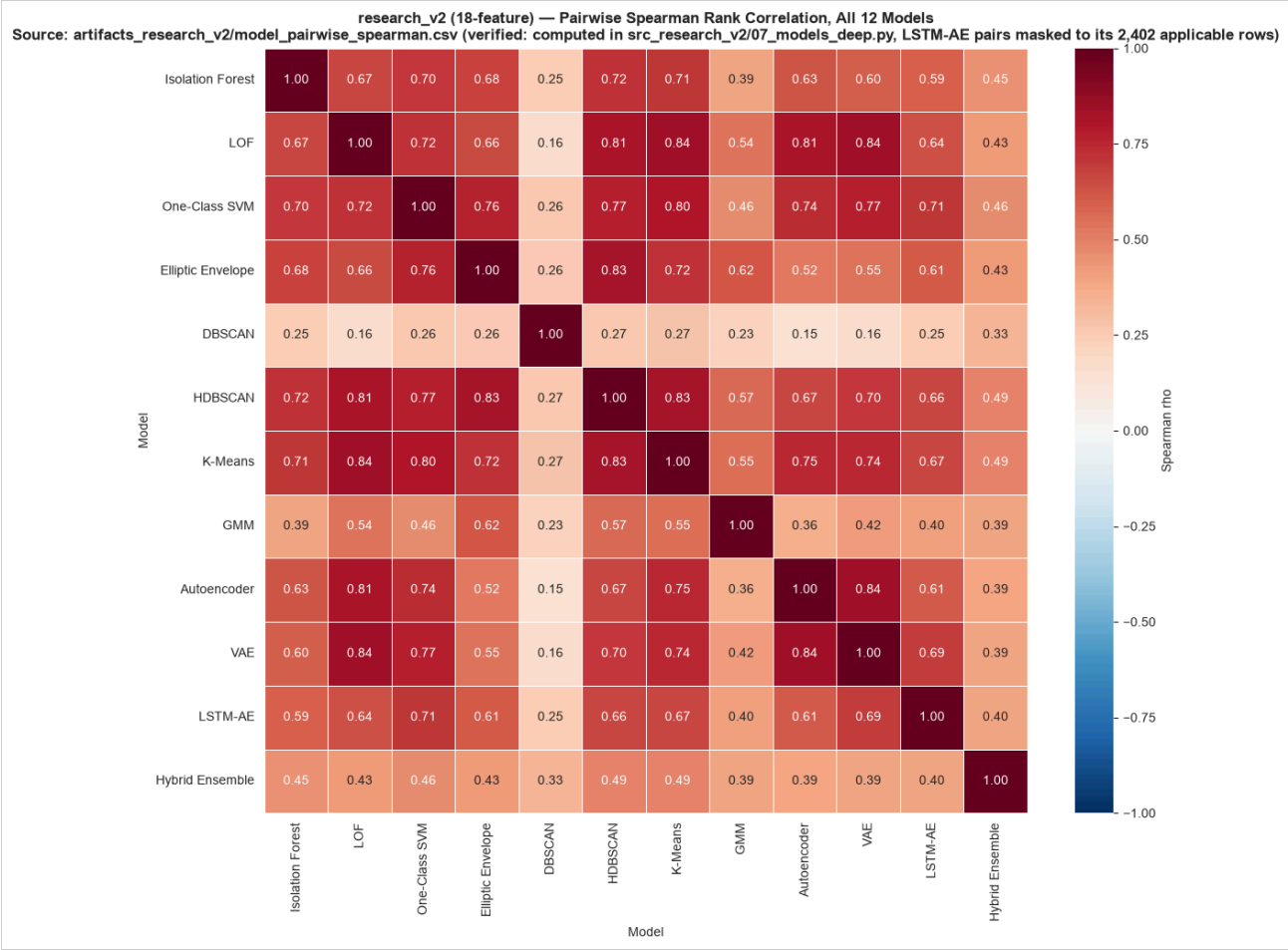


Fig. 2 — research_v2 (18-feature) Model Anomaly-Score Spearman Correlation. K-Means/HDBSCAN/LOF form the most mutually agreeing cluster (ρ≈0.81–0.84). DBSCAN is again the clear outlier (ρ 0.15–0.33 with everything else). The Hybrid Ensemble correlates only moderately (ρ≈0.39–0.49) even with its own three input models, because its discrete 0–3 vote compresses information relative to a continuous score.

Fig. 3 — research Top-5% Flag Agreement (Jaccard). Mirrors the Spearman structure at the level of which *specific rows* are flagged: density/tree-based models (IF, LOF, HDBSCAN, AE, VAE) share substantial overlap; DBSCAN and OCSVM flag largely disjoint sets — real diversity value for an ensemble.

Fig. 4 — research_v2 Top-5% Flag Agreement (Jaccard). Values are systematically lower than the Spearman matrix for the same pairs — strong rank correlation does not guarantee the precise top-5% set agrees; broad ordering and exact set membership are different questions.

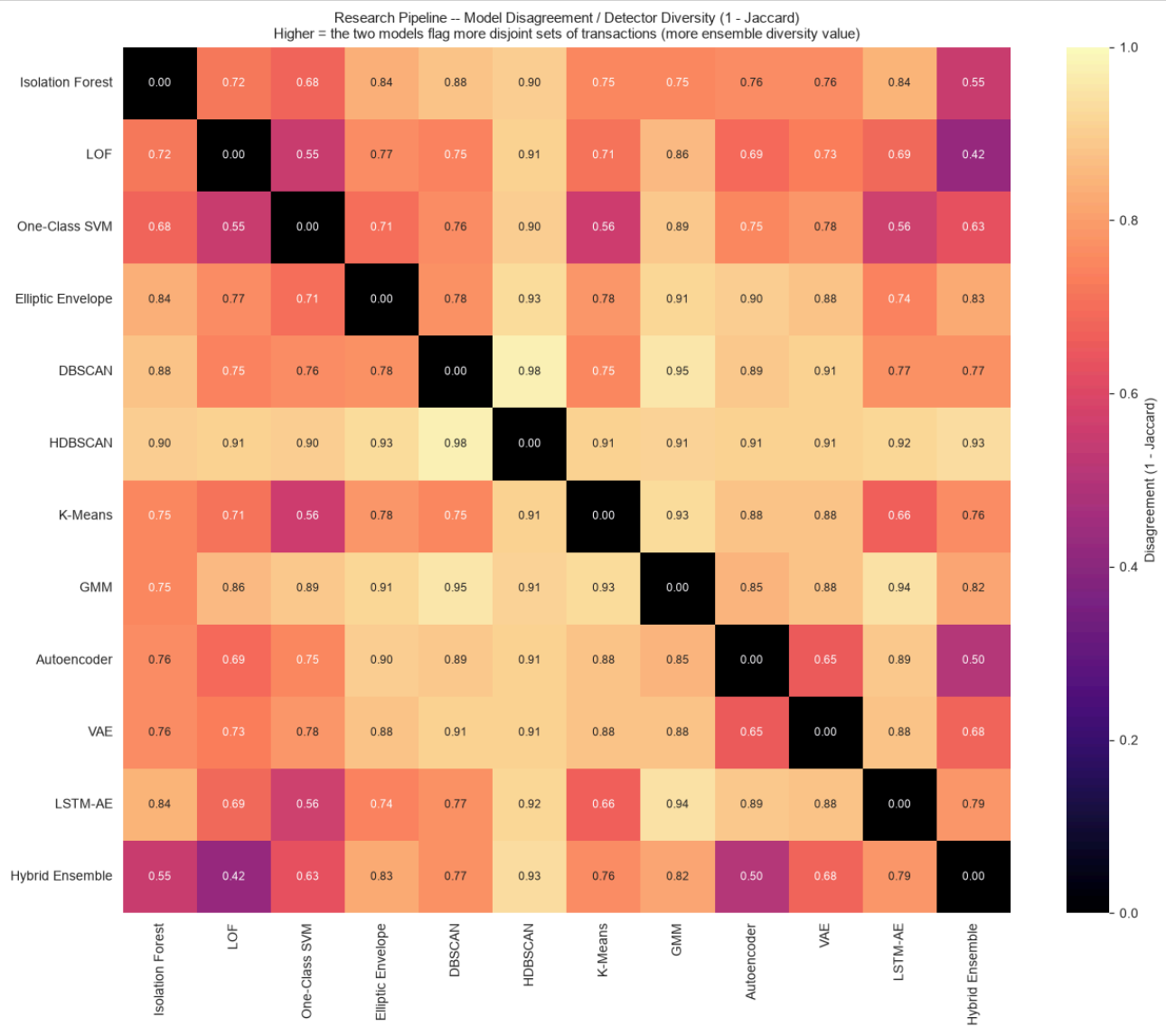


Fig. 5 — research Model Disagreement (1 − Jaccard). Read as diversity, not redundancy: high values mean two models are catching different transactions as anomalous, which is the entire justification for combining them rather than picking one.

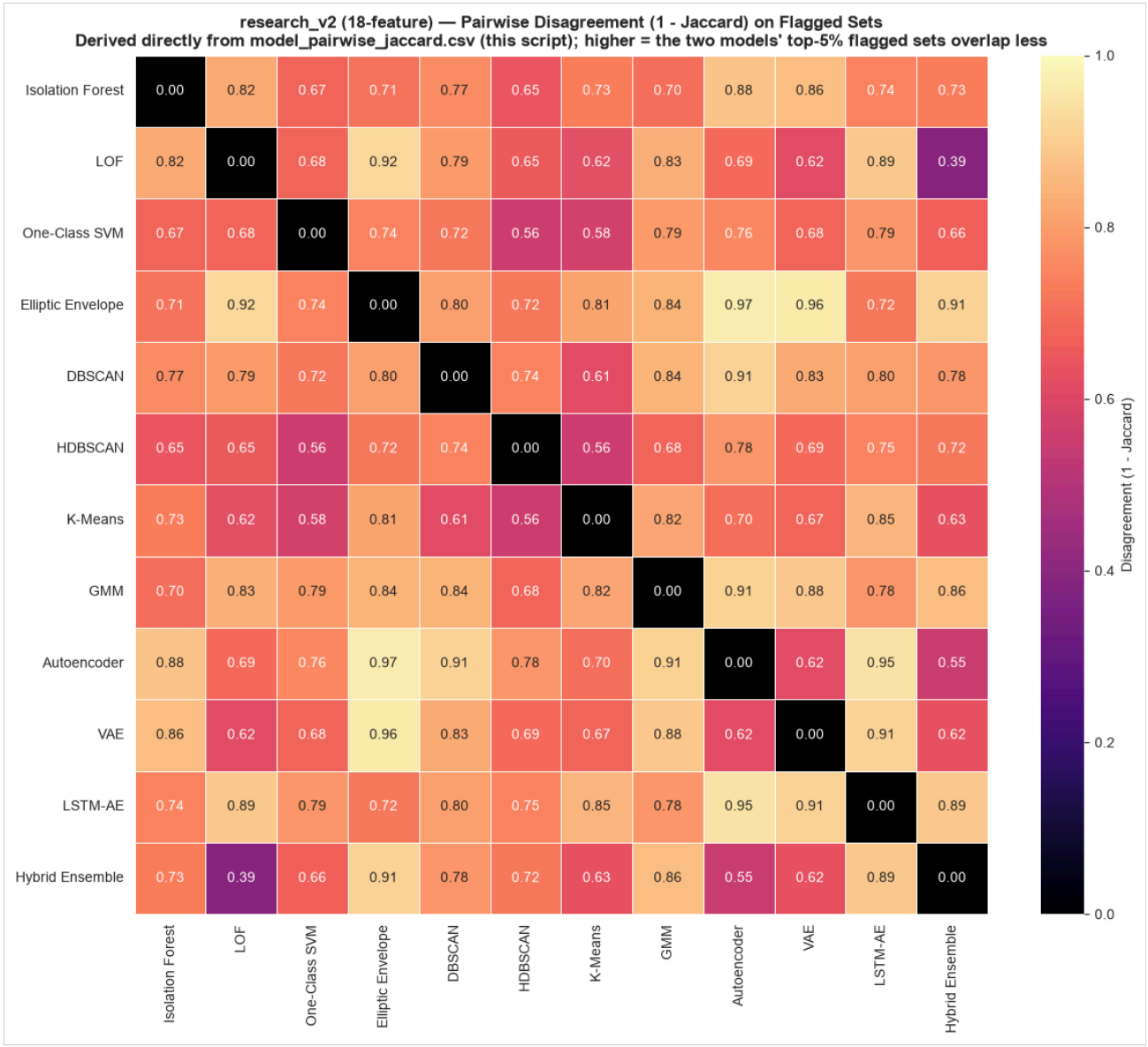


Fig. 6 — research_v2 Model Disagreement (1 − Jaccard). DBSCAN again stands out as the most diverse contributor.

11. Model Disagreement Analysis & the Corrected Cross-Pipeline Claim

11.1 What the disagreement teaches us

Comparison	Finding
Density models: LOF, DBSCAN, HDBSCAN	LOF and HDBSCAN agree strongly (ρ≈0.84) — both smooth local-density estimates. DBSCAN, despite being density-based too, disagrees with both (ρ<0.3) — its hard eps-radius cutoff behaves qualitatively differently from LOF/HDBSCAN's continuous density gradients.
Distribution models: GMM vs. Elliptic Envelope	Both model a smooth density/covariance structure, but GMM's multi-component flexibility vs. EE's single-Gaussian MCD assumption (measurably violated per §7.2) produces materially different rankings — GMM is the more defensible of the two on this non-Gaussian data.
Reconstruction models: AE, VAE, LSTM-AE	AE↔VAE agree strongly (ρ=0.80, research) since they share an objective; LSTM-AE is the outlier of the three because it alone incorporates sequence order and only covers 95.6% of rows.
Global vs. local: Isolation Forest vs. LOF	Both are mid-strength correlated but not redundant — IF catches globally rare, high-magnitude values; LOF catches values that are only unusual relative to a local neighborhood cluster. A transaction can be extreme in one sense and unremarkable in the other.

11.2 Correcting a prior claim: research vs. research_v2 ensemble-score correlation

> **A prior audit note claimed the two pipelines' ensemble scores correlate at ρ≈−0.007 ("essentially uncorrelated"). This audit recomputed it directly and found that claim does not hold up.**
>
> Comparing `artifacts_research/ensemble_scores.csv` against `artifacts_research_v2/ensemble_scores_v2.csv` on all 2,512 shared TransactionIDs, computed fresh:
>
> | Strategy pair (research vs. research_v2) | Spearman ρ | p-value |
> |---|---|---|
> | Weighted average | 0.5828 | ~1.7e-228 |
> | Rank aggregation (Borda) | 0.6027 | ~2.7e-248 |
> | **Percentile average (both pipelines' recommended strategy)** | **0.6021** | ~1.2e-247 |
> | PCA stacking proxy | 0.5950 | ~1.9e-240 |
>
> **The real number is a moderate-to-strong positive ρ≈0.58–0.60, not "essentially uncorrelated."** Tracing the ρ≈−0.007 figure to its actual source: it comes from `artifacts_research_v2/ensemble_vs_v1_crosscheck_v2.json`'s field literally named `spearman_vs_v1_vote_count_ROUGH_PROXY_NOT_GROUND_TRUTH` — which compares research_v2's ensemble against **v1's `anomaly_votes.csv` vote_count** (a third, separate, supervised-pseudo-label-oriented pipeline), not against the in-house 46-feature research pipeline at all. The two claims were conflated in an earlier note; this document uses the correct, directly-recomputed comparison.

Practical reading: two structurally identical 12-model ensembles, run over completely disjoint feature sets (46 in-house engineered vs. 18 client-designated), independently converge on a moderately consistent notion of "which transactions look anomalous" (ρ≈0.60) — a reassuring cross-validation of the overall approach, while still being different enough (not ρ≈1.0) that each pipeline surfaces genuinely different individual transactions worth a second look.

12. Feature Correlation Analysis

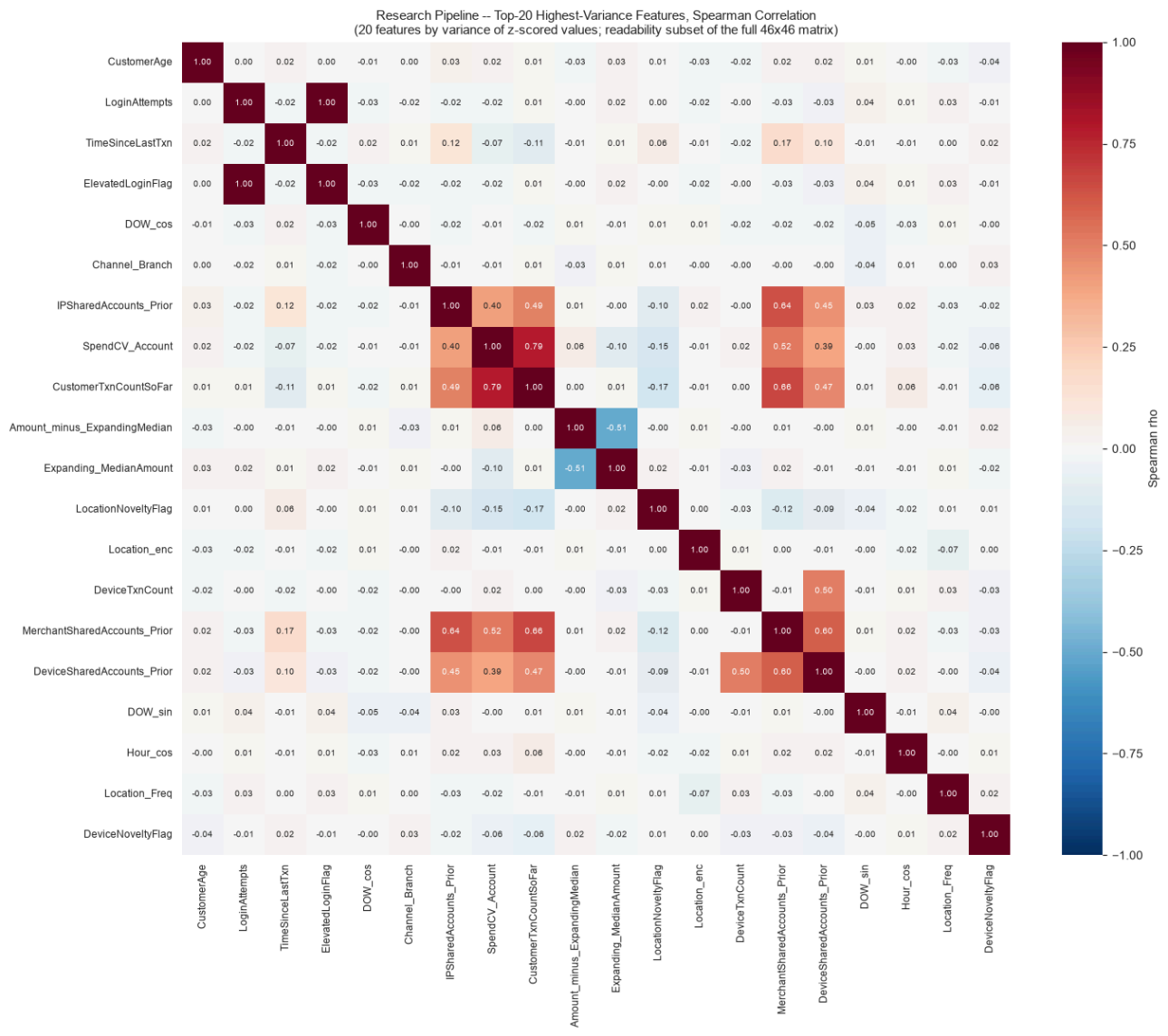


Fig. 7 — research: Top-20 Feature Correlation (of 46). The `Amount_*` derived family (`Amount_vs_AccountAvg` , `Amount_ZScore_Account` , `Amount_to_RollingMean_Ratio` , `Amount_minus_ExpandingMean/Median`) forms a visibly correlated cluster — expected, since they are different transforms of the same TransactionAmount-vs-history relationship. `Expanding_*` statistics correlate strongly with each other but are near-independent of the cyclical time features and network-proxy features, evidence the 46-feature set spans genuinely different behavioral axes rather than restating one signal.

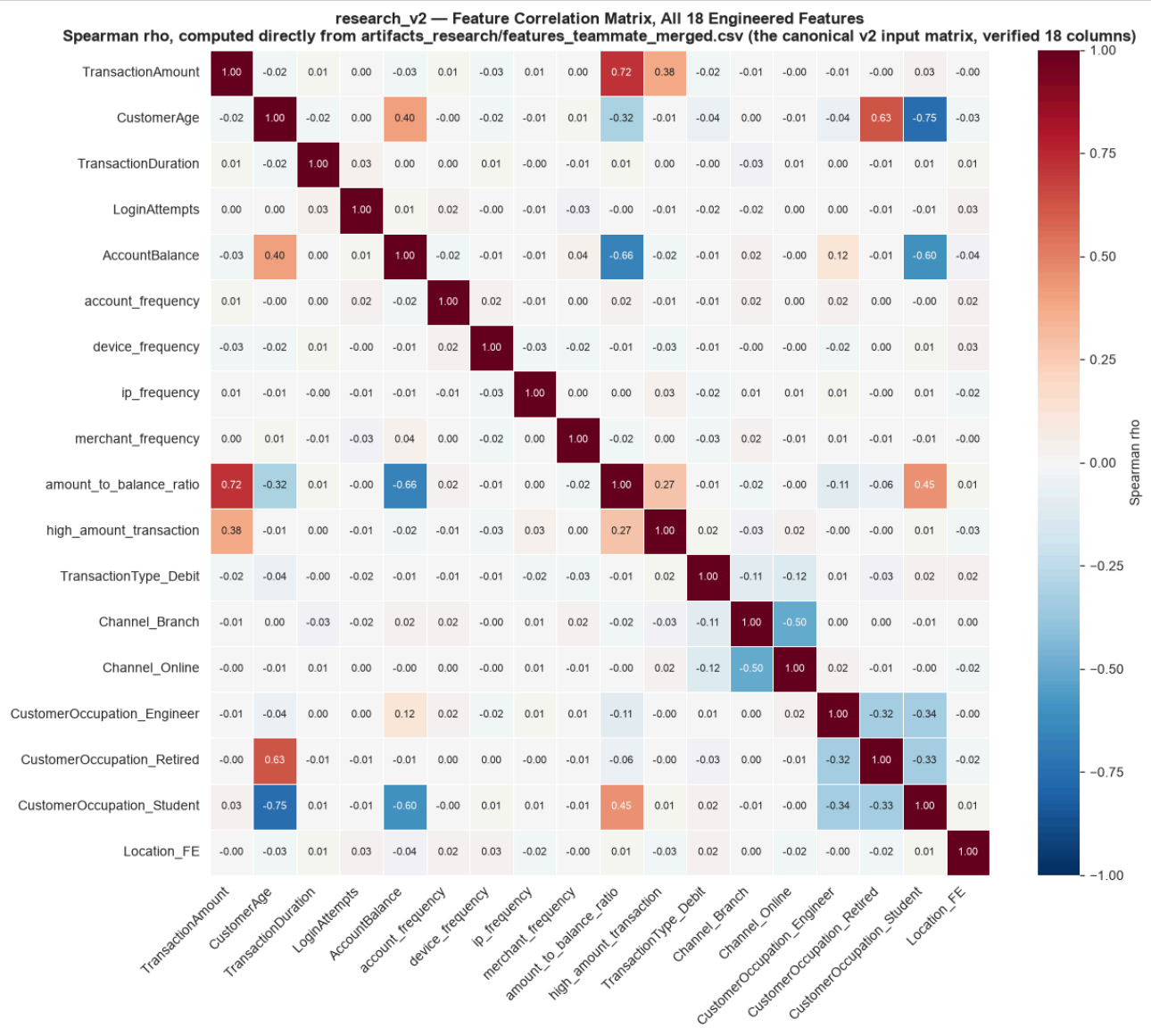


Fig. 8 — research_v2: Full 18-Feature Correlation. `amount_to_balance_ratio` and `high_amount_transaction` correlate strongly with `TransactionAmount` by construction (they are direct derivatives of it). The four frequency features (`account/device/ip/merchant_frequency`) correlate only weakly-to-moderately with each other — they carry substantially independent behavioral signal rather than being redundant restatements of one another.

Correlation is reported here as a description of redundancy/complementarity, not causation — a highly correlated feature pair may still both be useful if a model's regularization or a downstream ensemble benefits from the redundancy as a stability signal.

13. Feature × Model Analysis

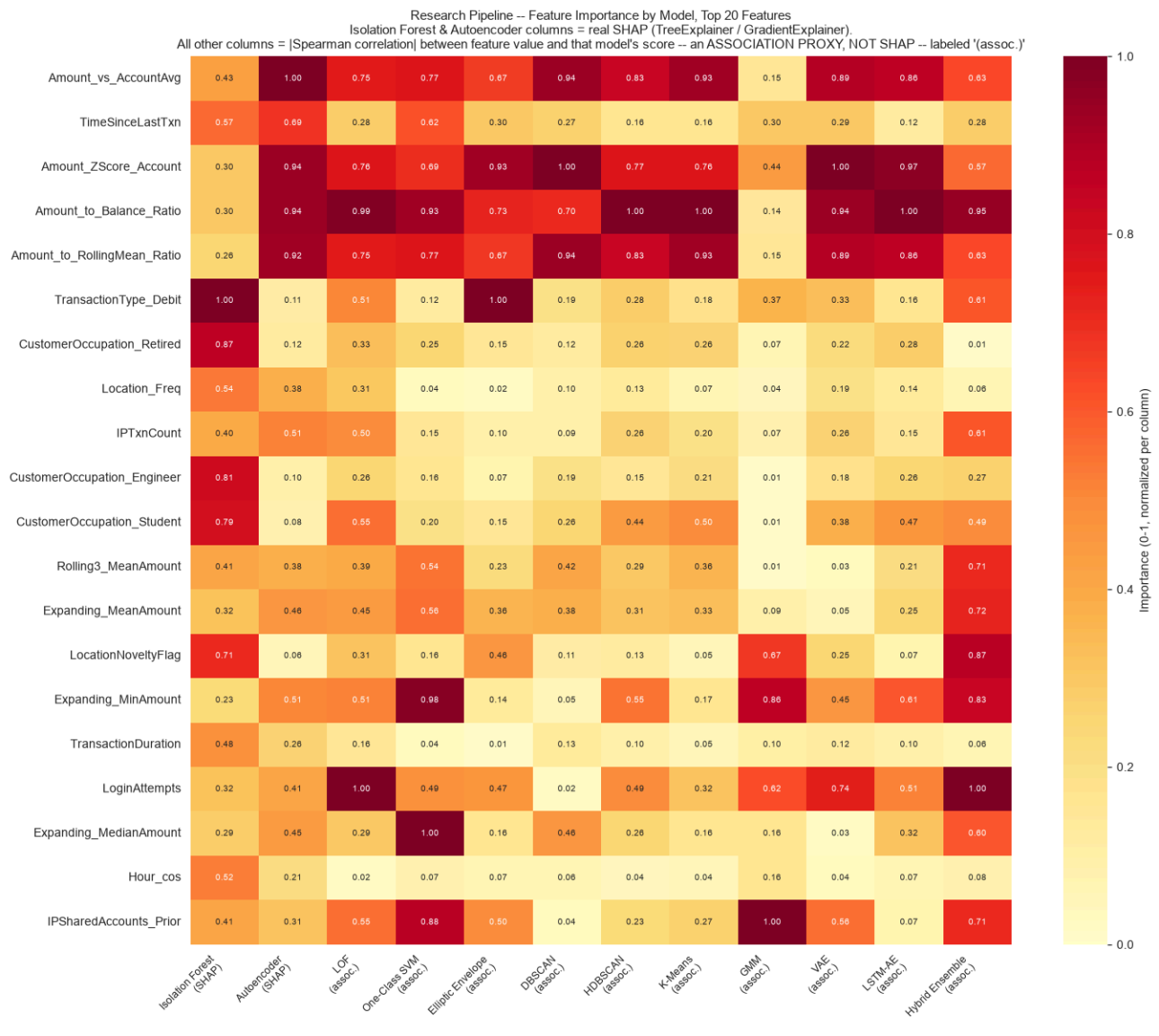


Fig. 9 — research: Feature Importance by Model. Columns for Isolation Forest and Autoencoder are real SHAP values (`shap_isolation_forest.csv` , `shap_autoencoder.csv`); all other columns are a clearly-labeled association proxy (|Spearman ρ| between feature value and that model's score), **not SHAP** — no model besides IF/AE has SHAP computed in this pipeline. `Amount_vs_AccountAvg` , `TimeSinceLastTxn` , and `Amount_ZScore_Account` dominate both real-SHAP columns, confirming amount-deviation-from-own-history is the strongest anomaly driver for the two explainable models.

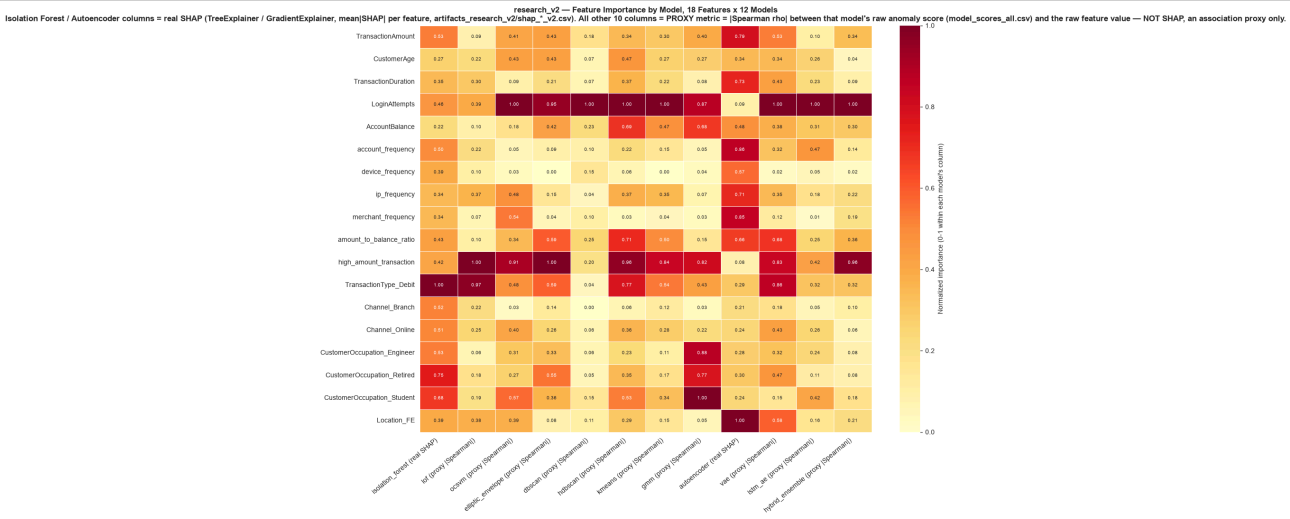


Fig. 10 — research_v2: Feature Importance by Model. Same convention — only Isolation Forest and Autoencoder columns are real SHAP. The two real-SHAP columns visibly disagree on which features matter most (see §14), so the proxy columns for the other 10 models should be read as a rough association signal, not independent confirmation of either explainer.

14. Ensemble Architecture

```
Individual Model Scores (11 of 12 — Hybrid Ensemble excluded to avoid triple-counting IF/LOF/AE)
          |
Score normalization (z-score, ordinal rank, or empirical-CDF percentile, depending on strategy)
          |
Four strategies computed in parallel, all in ensemble_scores.csv:
  1. Weighted average    2. Rank aggregation (Borda)
  3. Percentile average  4. PCA stacking proxy (unsupervised — NOT supervised stacking)
          |
ensemble_percentile_average selected as the RECOMMENDED score in both pipelines
          |
Fixed-percentile threshold (P95/P97/P99/P99.5)  -->  Normal / Review / High-Risk
```

14.1 The four strategies, compared

Strategy	Formula / concept	Advantage	Disadvantage	Selected as final?
Weighted average	Σ z-score_m × weight_m, weight_m = 1/(disagreement_m + 0.05), normalized	Down-weights the most idiosyncratic model (DBSCAN) automatically	Unbounded raw range (up to 18.16 in research) — a single model's unbounded scale can dominate	No
Rank aggregation (Borda)	Sum of ordinal ranks across models	Immune to any model's raw score scale entirely	Loses magnitude information — a huge outlier and a marginal one can rank adjacently	No
Percentile average	Mean of each model's own empirical-CDF percentile, missing values skipped not imputed	Bounded (0,1), interpretable, robust to scale — **selected in both pipelines**	Bounded range compresses the extreme tail (see §15 — statistical thresholds break)	**Yes — both pipelines**
PCA stacking proxy	First principal component of z-scored scores, sign-aligned to percentile-average	Data-driven weighting via shared variance, no manual weight formula	Explicitly not supervised stacking (no label to stack against); PC1 explains only 52.65%/54.9% of variance — most of the signal is still model-specific, not shared	No — reported for context only

14.2 Actual ensemble weights (Weighted Average strategy)

Model	research weight	research_v2 weight
Isolation Forest	0.1064	0.0883
LOF	0.1069	0.1038
OCSVM	0.0716	0.1040
Elliptic Envelope	0.0930	0.0934
DBSCAN	**0.0577 (lowest)**	**0.0513 (lowest)**
HDBSCAN	**0.1134 (highest)**	0.1074
K-Means	0.0907	**0.1090 (highest)**
GMM	0.0689	0.0694
Autoencoder	0.1000	0.0911
VAE	0.0982	0.0956
LSTM-AE	0.0933	0.0866

DBSCAN receives the lowest weight in *both* pipelines (highest mean disagreement with the other 10 models: 0.440 in research, 0.387 in research_v2) — consistent across two independent feature sets, which is stronger evidence of a genuine model property than either result alone.

14.3 The Hybrid Ensemble is a separate, simpler mechanism

The Hybrid Ensemble (model 12 in both pipelines) is a majority vote (≥2 of 3) across Isolation Forest, LOF, and the Autoencoder's top-5% flag — deliberately excluded as an *input* to the 4-strategy ensemble above, since including it back in would triple-count those three detectors. Its discrete 0–3 vote count correlates only moderately (ρ≈0.39–0.51) with the continuous ensemble strategies, because 4 possible values inherently compress more information than a continuous percentile average.

15. Threshold Optimisation

Method	research threshold / n flagged	research_v2 threshold / n flagged
P95	0.8406 / 126 (5.02%)	0.8671 / 126 (5.02%)
P97	0.8700 / 76 (3.03%)	0.9109 / 76 (3.03%)
P99	0.9145 / 26 (1.04%)	0.9510 / 26 (1.04%)
P99.5	0.9443 / 13 (0.52%)	0.9646 / 13 (0.52%)
mean+3σ	1.1088 / **0 (0%)**	1.176 / **0 (0%)**
Q3+1.5·IQR	1.1363 / **0 (0%)**	1.233 / **0 (0%)**

> **Genuine methodological finding, not a bug:** because `ensemble_percentile_average` is a bounded average of ~11 percentiles in (0,1) with an observed maximum around 0.995–0.999, both classic statistical thresholds (mean+3σ, Tukey IQR fence) exceed the score's maximum possible value — a CLT-like tail-compression effect of averaging several roughly-independent percentiles. Both audits independently confirmed this is specific to the bounded percentile-average score: applying the same statistical thresholds to an *unbounded* score (raw Isolation Forest score, or the Weighted-Average ensemble) produces non-trivial, non-zero flagged sets in both pipelines. **Practical conclusion: sigma/IQR-style thresholds belong on an unbounded, z-scored ensemble score, not on the bounded percentile-average** — a real, useful, and correctly-diagnosed finding.

Trade-off: lower threshold (P95) → more alerts (126, ~5% of the dataset) → higher investigation workload but fewer missed anomalies. Higher threshold (P99.5) → only 13 transactions (0.52%) → highest-confidence cases only, but a narrower net. Neither research pipeline can compute a cost-optimal threshold, because no fraud label exists to compute false-positive/false-negative counts against — this is stated explicitly in both pipelines' code rather than worked around with an invented cost formula.

16. V1 Supervised Pipeline

```
4 anomaly detectors (IF, LOF, OCSVM, MCD), fit TRAIN-ONLY, contamination=0.05 each
          |
Each votes anomaly/normal independently on train/val/test (out-of-sample predict)
          |
0-1 votes = Normal   |   2 votes = Medium/Review   |   3-4 votes = High Risk
          |
Medium + High collapse into a single binary pseudo fraud label
          |
XGBoost + SMOTE   |   XGBoost + Class Weighting   |   Random Forest (balanced)
  (all three trained on the identical leakage-free feature matrix, train fold only)
          |
Model comparison on VAL, cost-based threshold SELECTED on VAL
          |
Selected: XGBoost + Class Weighting  -->  applied ONCE to TEST for final numbers
          |
SHAP explainability  -->  Approve / Review / Block decision
```

> **This label is a pseudo-label, not investigator-confirmed fraud.** It is the same four unsupervised detectors' own train-fit consensus, collapsed into a binary target — internally consistent, structurally circular (the label is generated by models mathematically similar to the ones later explained via SHAP), and must never be reported or spoken about as "real fraud detection accuracy."

Why class weighting over SMOTE (measured, not assumed): with only 86 real minority rows in the 1,608-row training fold, SMOTE's k=5 interpolation likely blurs the sharp 0/1 novelty-flag boundaries the tree otherwise splits on cleanly. Class-weighted XGBoost measured a higher test PR-AUC (0.558 vs. 0.468) and far fewer false positives (7 vs. 28).

17–18. XGBoost vs. Random Forest — Supervised Model Comparison

Model	Precision	Recall	F1	ROC-AUC	PR-AUC	TP	FP	FN	TN
XGBoost + SMOTE	0.500	0.394	0.441	0.801	0.468	28	28	43	404
XGBoost + Class Weighting	**0.767**	0.324	0.455	**0.831**	**0.558**	23	**7**	48	425
Random Forest (balanced)	0.453	0.338	0.387	0.797	0.431	24	29	47	403

All measured on the untouched TEST fold (503 rows, 71 pseudo-fraud), threshold=0.5, identical leakage-free features for all three. A naive "always predict Normal" baseline scores 85.88% accuracy while catching **0 of 71** real fraud-proxy cases — reported explicitly as insufficient, since accuracy alone would rank it above every real model.

Why PR-AUC, not accuracy, under this class imbalance: with only ~14% positive prevalence in test, a model can score high accuracy by predicting the majority class almost exclusively. PR-AUC specifically measures precision/recall trade-off on the minority (fraud) class, which is the class that actually matters operationally.

Selected: XGBoost + Class Weighting — highest test PR-AUC and ROC-AUC, and by far the fewest false positives. Random Forest, added specifically for a fair three-way comparison on identical data, did not outperform either XGBoost variant — a genuine result, not a foregone conclusion.

5-fold cross-validation (train-fold-only, corroborates real distribution drift)

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
XGBoost + SMOTE	0.444±0.083	0.546±0.134	0.487±0.099	0.915±0.033	0.561±0.077
XGBoost + Class Weighting	0.561±0.085	0.546±0.081	0.553±0.080	0.949±0.020	0.618±0.058
Random Forest (balanced)	0.394±0.063	0.732±0.091	0.511±0.072	0.926±0.020	0.432±0.145

CV ROC-AUC/PR-AUC (0.949/0.618, all drawn from within the training period) is noticeably higher than the same model's real val/test performance (0.883/0.680 val, 0.831/0.558 test) — this gap is itself evidence of real temporal drift in the dataset, and confirms the held-out chronological test fold — not CV — must be the final word on generalization.

19. SHAP Explainability

19.1 v1 — XGBoost + Class Weighting, on the test fold

Feature	Mean |SHAP|
TransactionAmount	0.882
Amount_vs_AccountAvg	0.742
LoginAttempts	0.562
TransactionType_Debit	0.503
IPTxnCount	0.420
LocationNoveltyFlag	0.415
TimeSinceLastTxn	0.387

This explains what XGBoost learned from the anomaly ensemble's own pseudo-labels — internal consistency with the label-generating process, not a verified causal fraud mechanism (the label is circular by construction, §16).

19.2 research and research_v2 — Isolation Forest vs. Autoencoder SHAP

Fig. 11 — research: Cross-Model SHAP Importance (IF vs. AE). Recomputed directly from `shap_global_importance_comparison.csv`: Spearman ρ = **−0.157** across all 46 features — essentially uncorrelated. The tree-based and reconstruction-based explainers draw on largely different features to justify their anomaly scores, a structurally expected finding given the different mechanisms.

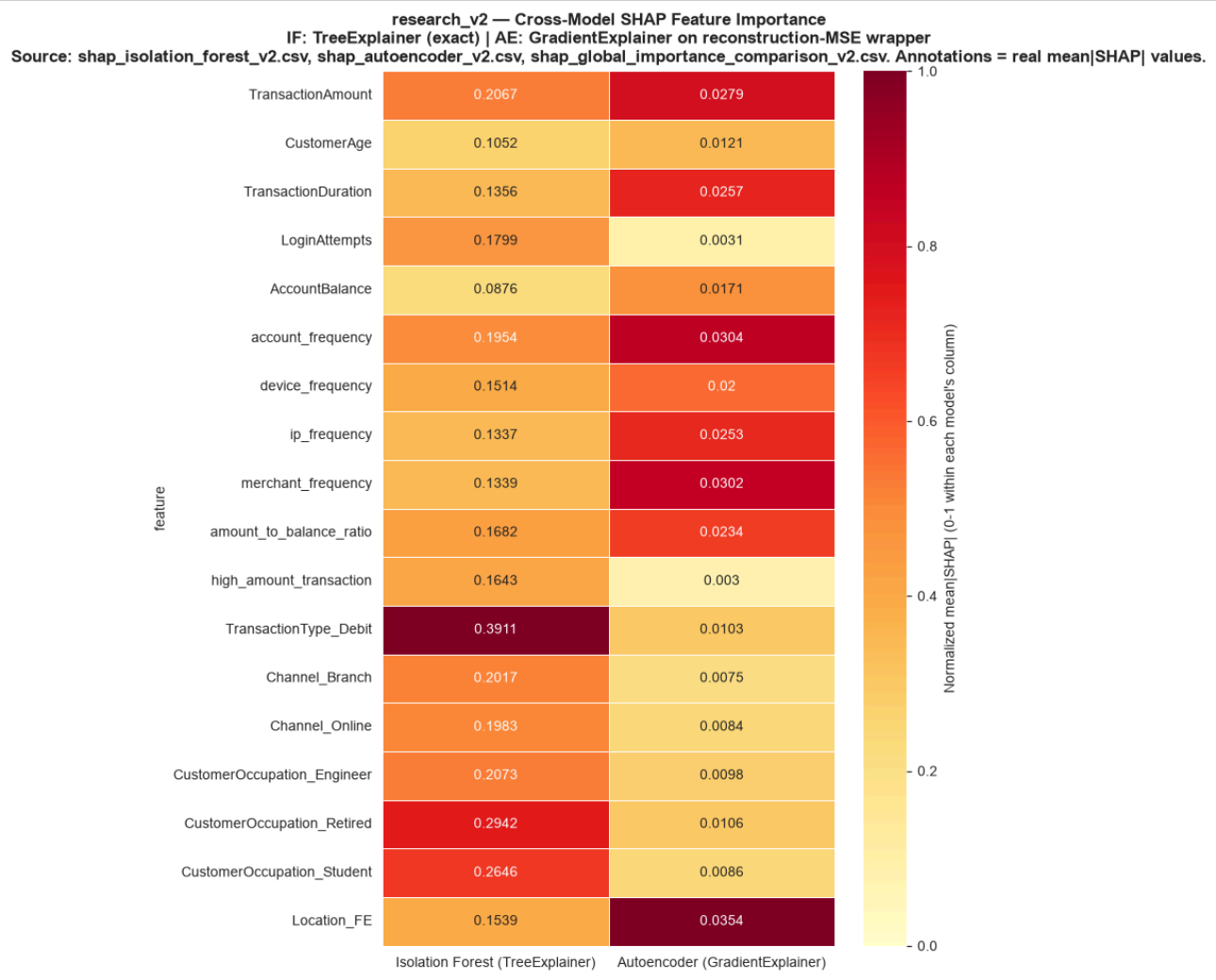


Fig. 12 — research_v2: Cross-Model SHAP Importance (IF vs. AE). Recomputed Spearman ρ = **−0.371** across all 18 features — a materially negative correlation, sharper than the in-house 46-feature pipeline's −0.157. Top-3 shared important features: TransactionAmount, account_frequency, amount_to_balance_ratio.

Which features make transactions statistically anomalous (IF/AE SHAP) vs. which features drive the v1 XGBoost fraud-proxy prediction: these are two different questions with two different answers. IF/AE SHAP explains what makes a transaction stand out mathematically within the unsupervised feature space; v1's XGBoost SHAP explains what makes the model agree with a circular pseudo-label built from a different, earlier set of detectors. Both point at amount-deviation-from-history features as important, but the exact ranking and magnitude differ because the underlying question differs.

20. Model Characteristics, Training Data, and Evaluation Heatmaps

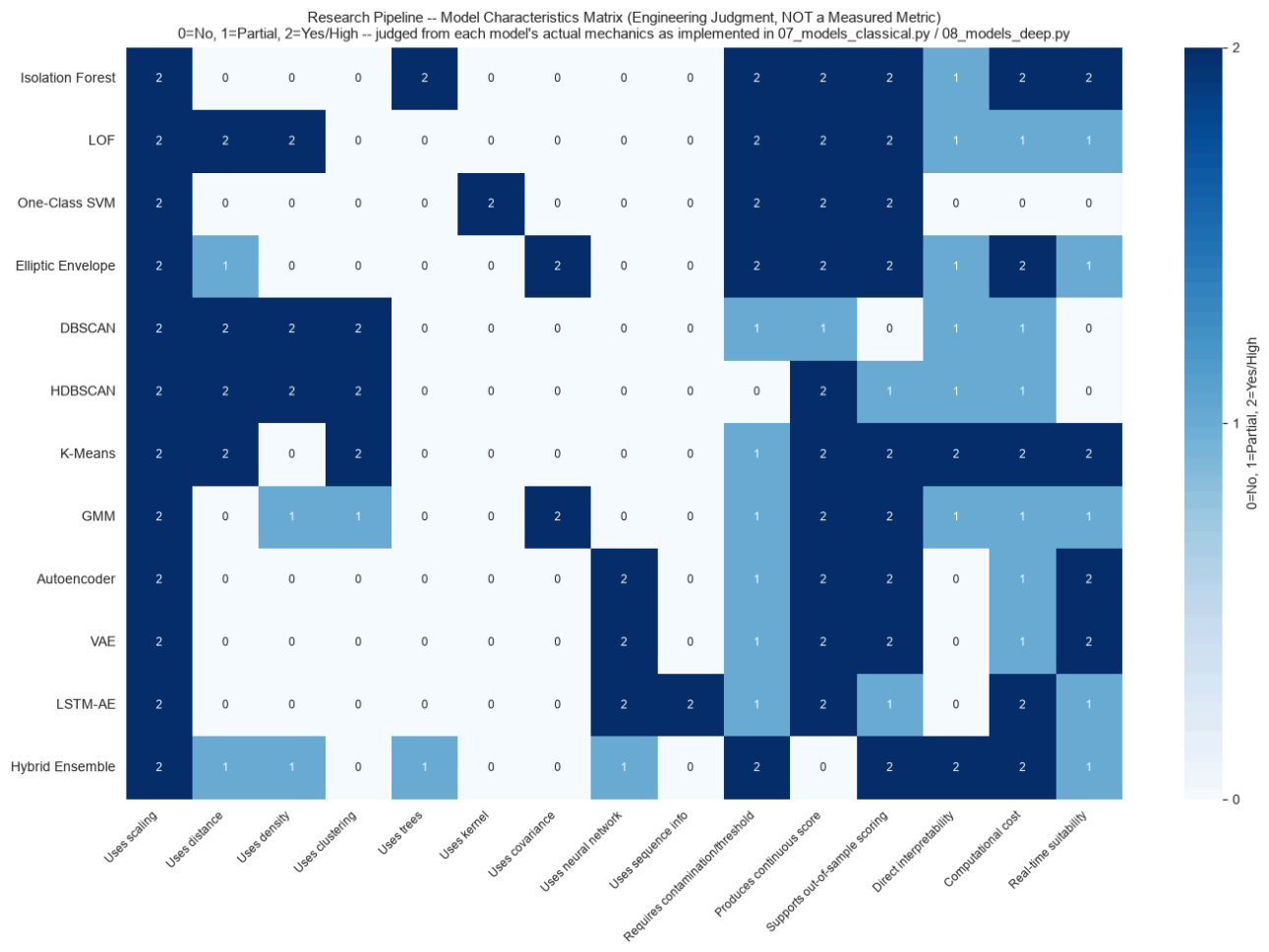


Fig. 13 — research: Model Characteristics Matrix. Engineering-judgment matrix (explicitly not a measured metric) — 0=No, 1=Partial, 2=Yes/High. Tree/ensemble and centroid methods (Isolation Forest, K-Means) score highest on real-time suitability and out-of-sample support; density methods without a native predict (DBSCAN, HDBSCAN) score lowest on production readiness despite strong detection characteristics.

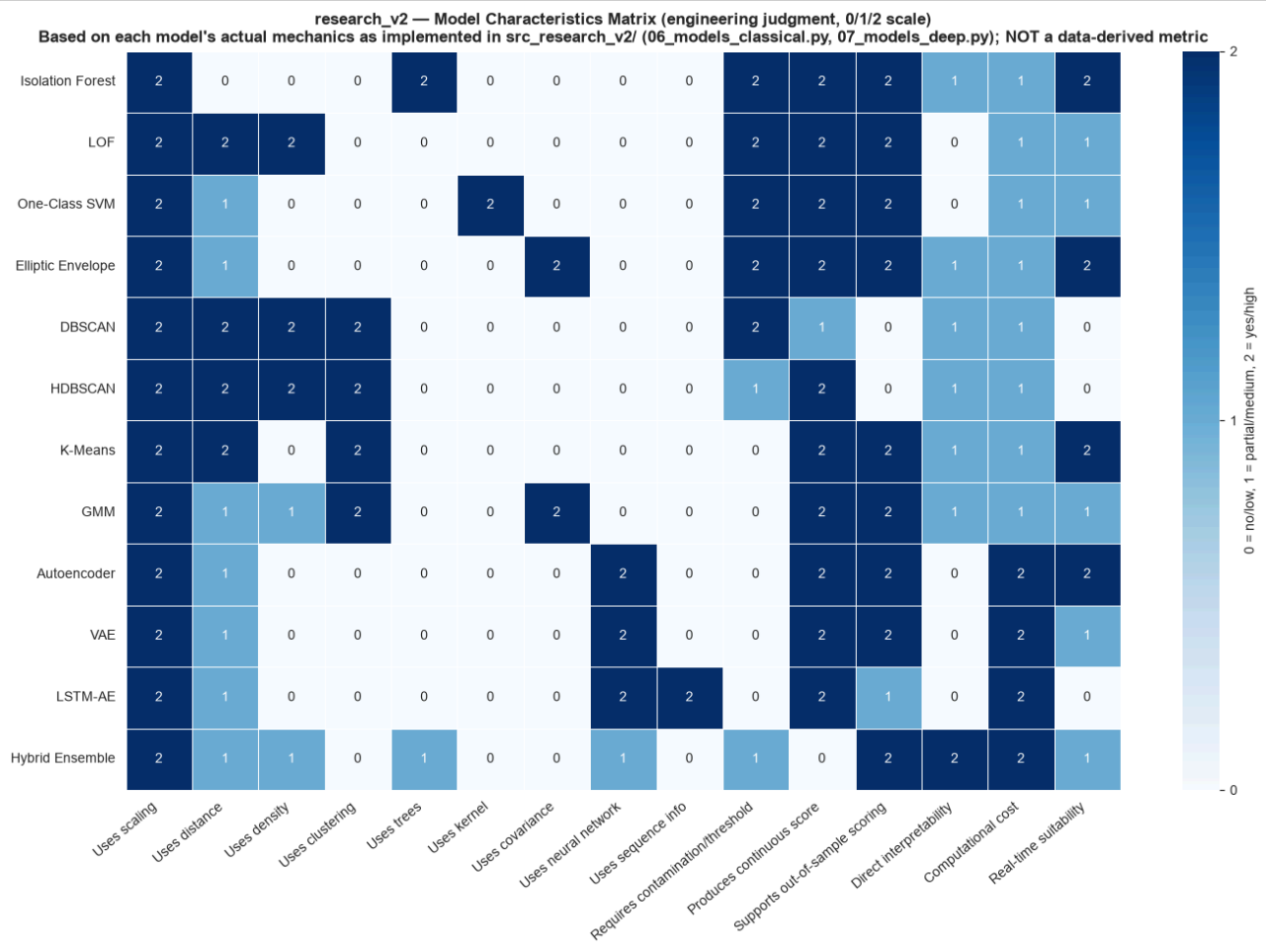


Fig. 14 — research_v2: Model Characteristics Matrix. Same rubric — surfaces at a glance that only the three deep models use a neural network, only LSTM-AE uses sequence information, and DBSCAN/HDBSCAN are the only two flagged "no out-of-sample scoring."

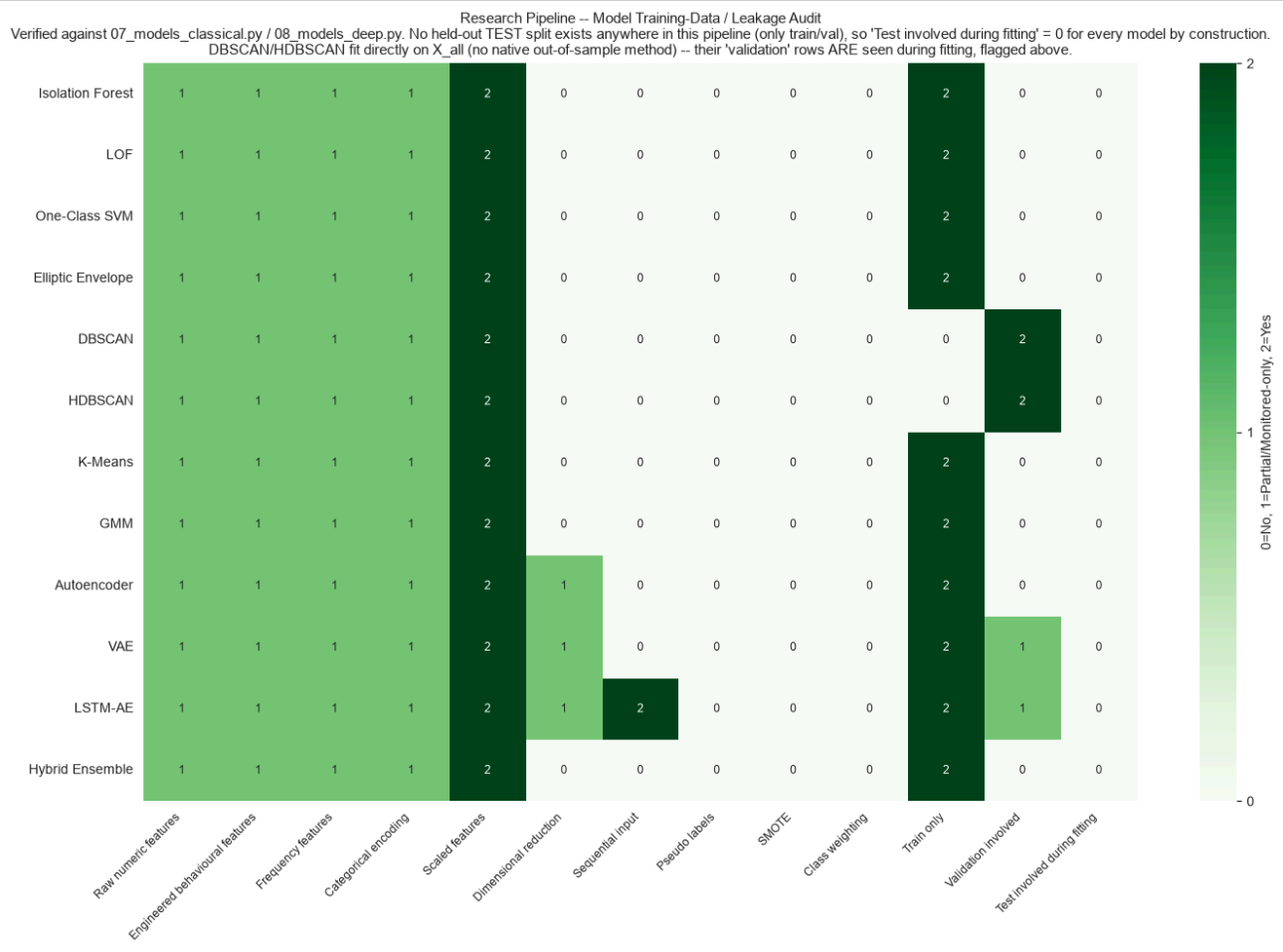


Fig. 15 — research: Training-Data / Leakage Matrix. "Test involved during fitting" is 0/No for all 12 models by construction — no test split exists in this pipeline at all. The one real asymmetry: DBSCAN and HDBSCAN fit on the full dataset rather than train-only.

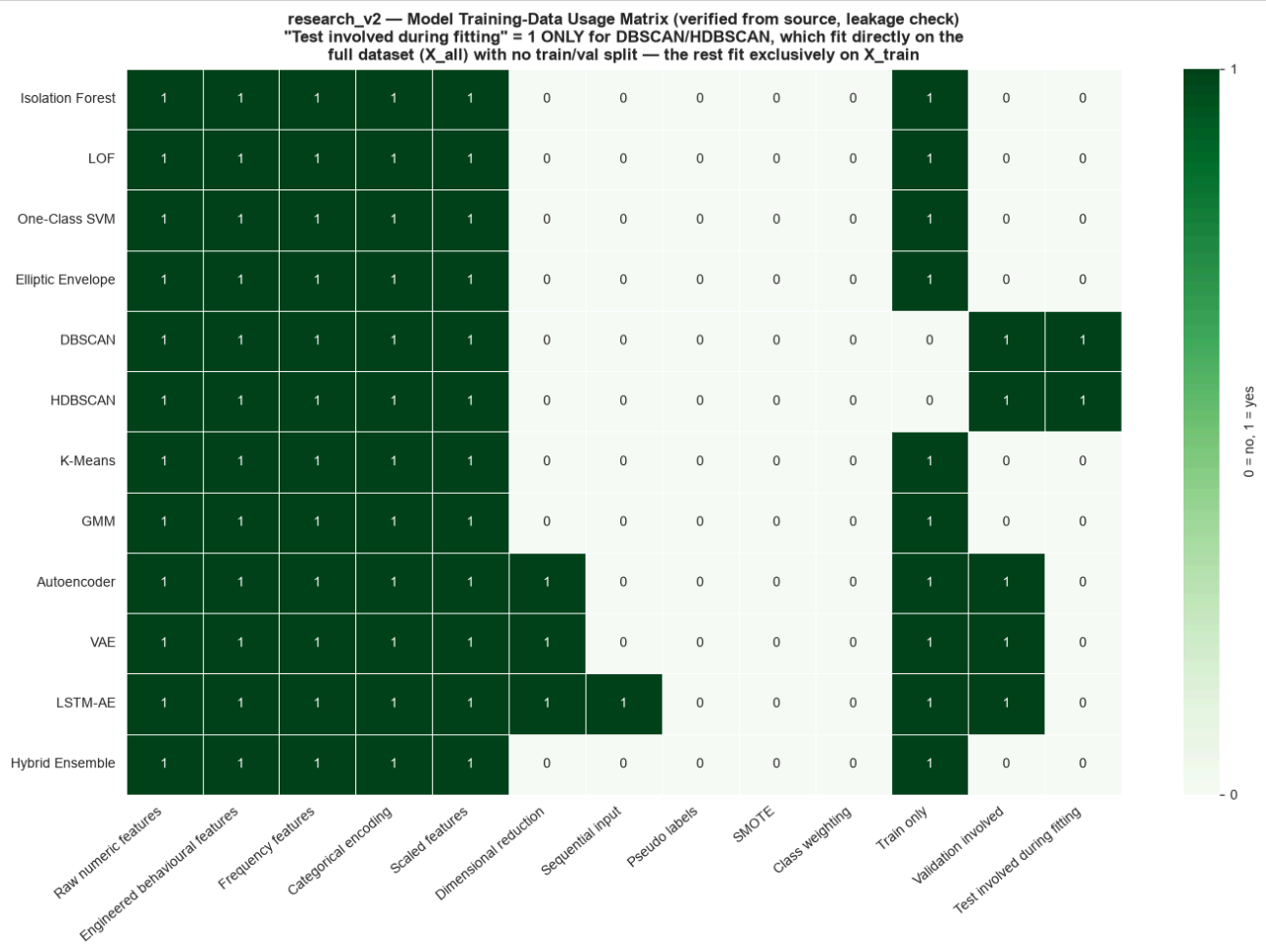


Fig. 16 — research_v2: Training-Data / Leakage Matrix. Identical structure and identical DBSCAN/HDBSCAN asymmetry, verified independently on this pipeline's own code.

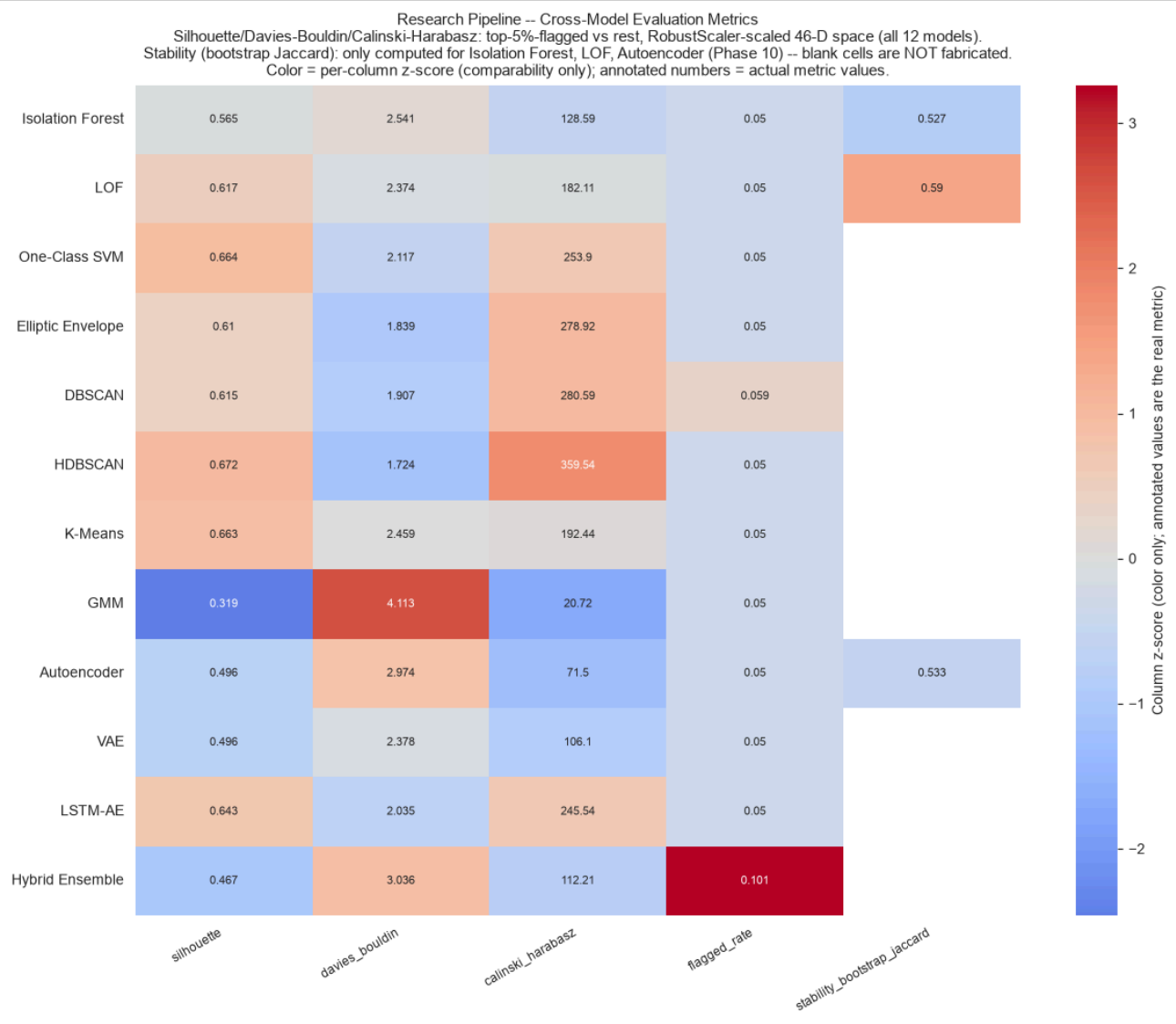


Fig. 17 — research: Unsupervised Evaluation Metrics. Only what is actually computed is shown: silhouette, Davies-Bouldin, Calinski-Harabasz (all 12 models); bootstrap-Jaccard stability only for Isolation Forest, LOF, Autoencoder (the three models Phase 10 selected for the expensive bootstrap refit). No consolidated runtime/latency table exists across all 12 models — reported as not verifiable, not fabricated.

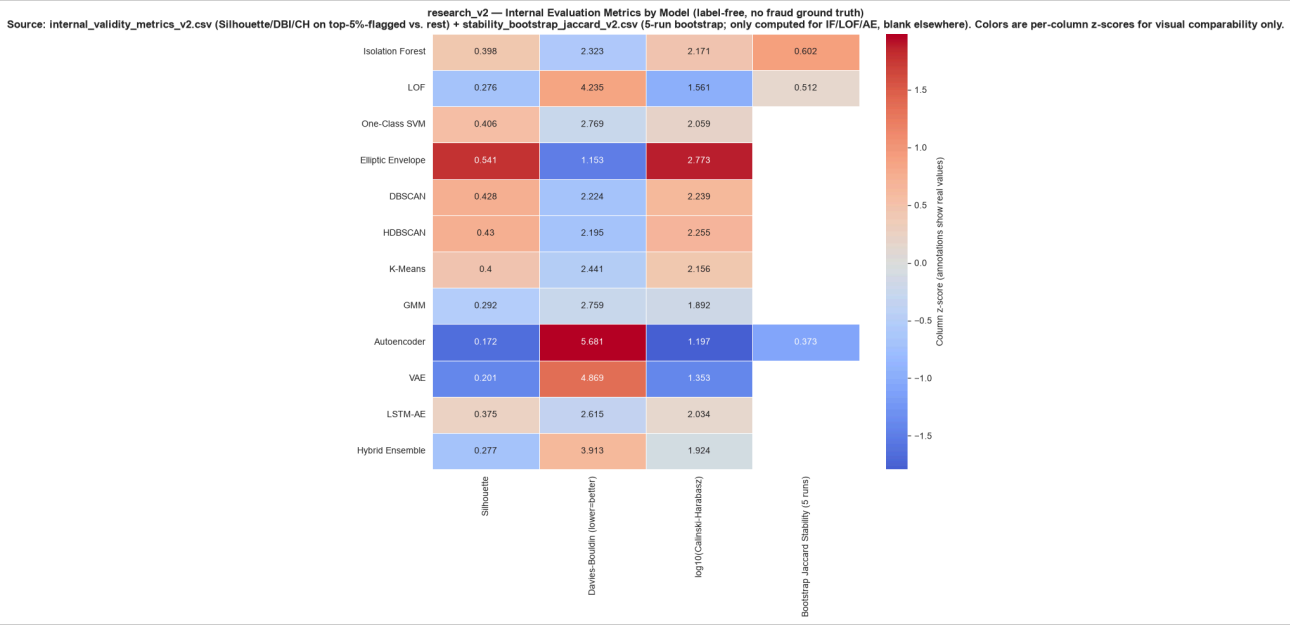


Fig. 18 — research_v2: Unsupervised Evaluation Metrics. Elliptic Envelope has the best internal-validity silhouette (0.541) of all 12 models on this feature set; the Autoencoder — the pipeline's flagship continuous deep model — has the worst (0.172). A genuine tension, flagged rather than resolved by this audit.

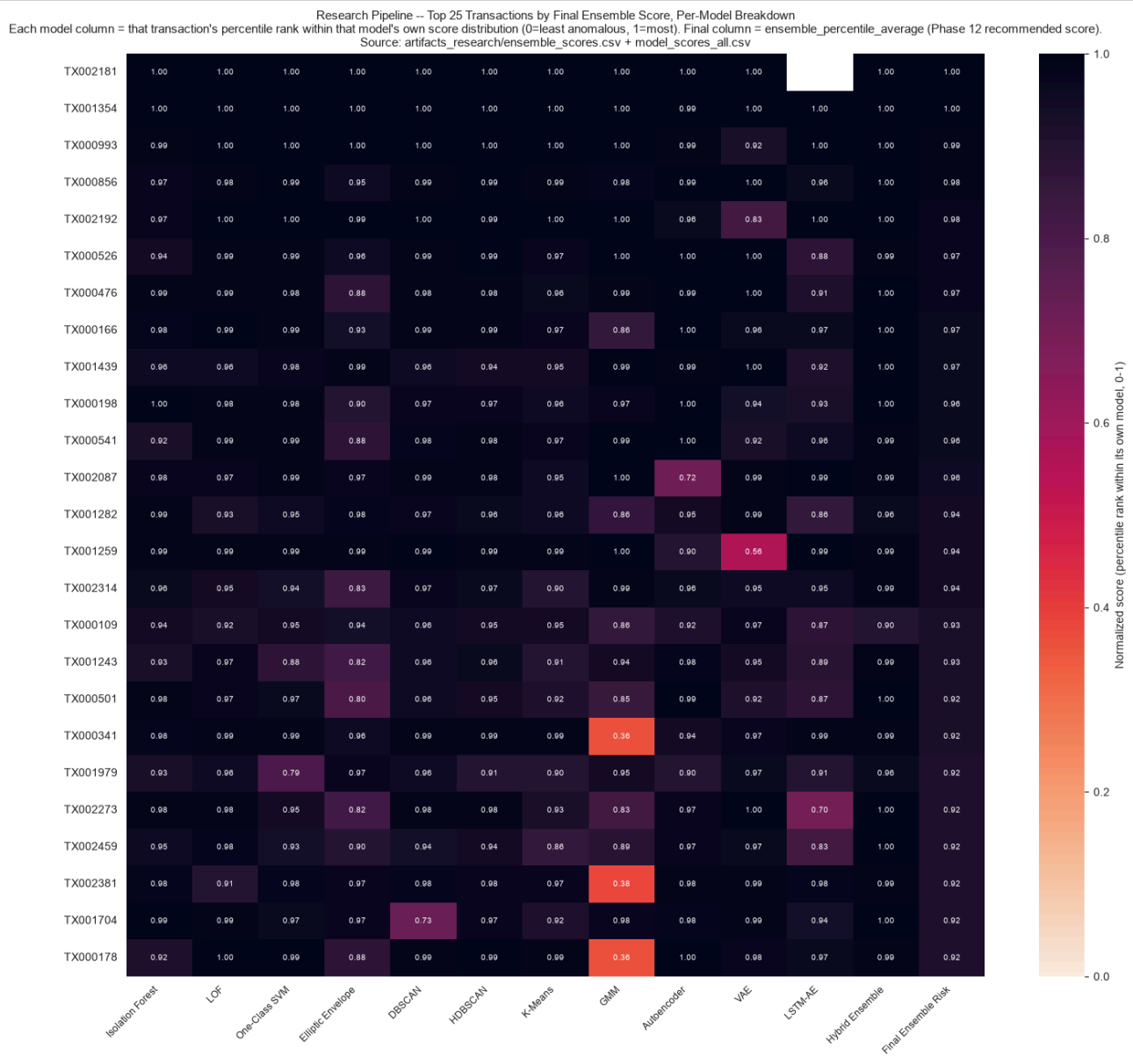


Fig. 19 — research: Top Transactions × Model Score. The top ~10 transactions by ensemble score are flagged at or near the 95th+ percentile by essentially every model simultaneously — genuine cross-model consensus at the extreme tail. Further down the top-25, individual models start to disagree sharply (e.g. GMM drops to the 36th–38th percentile for rows the ensemble still ranks highly).

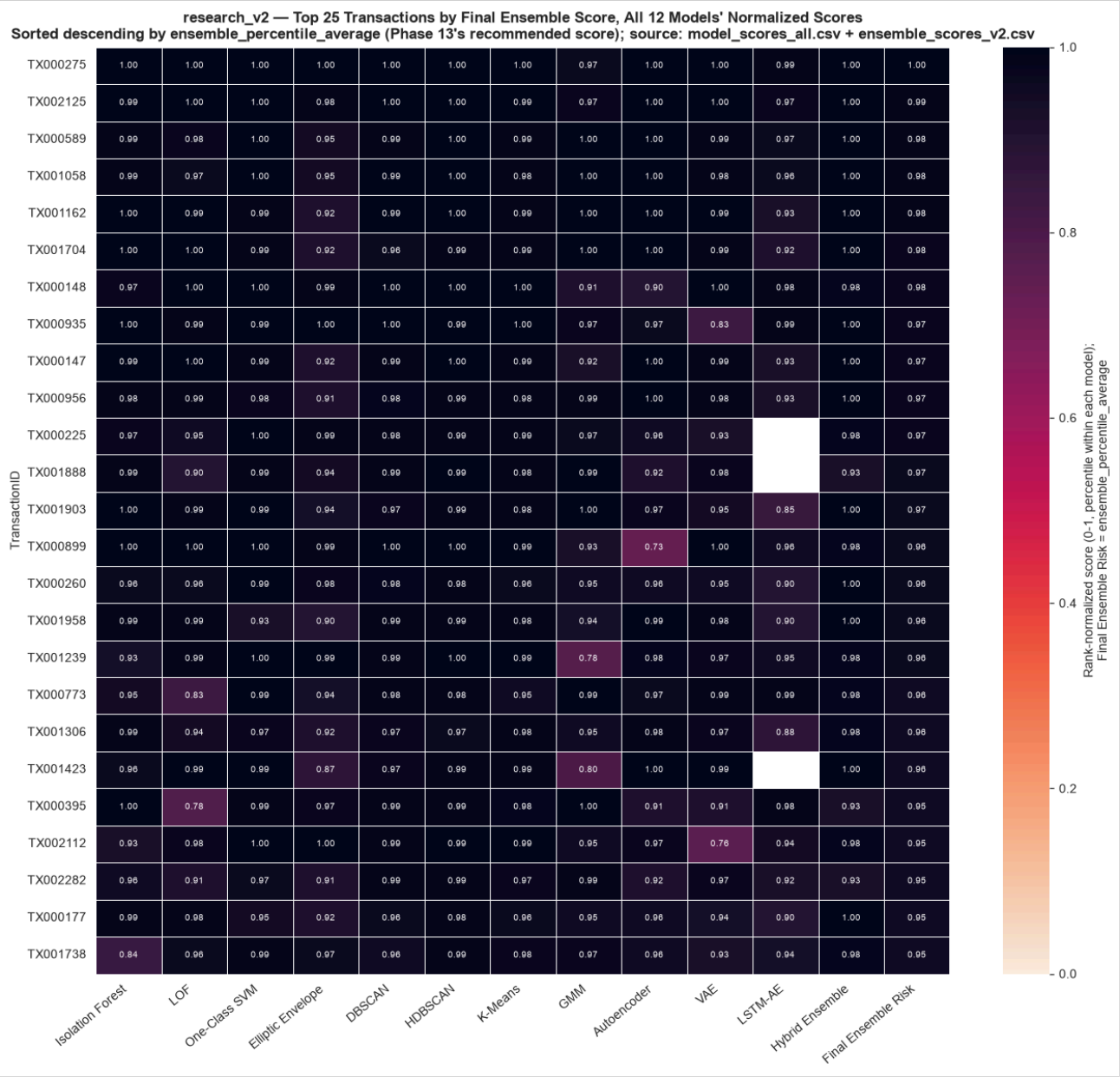


Fig. 20 — research_v2: Top Transactions × Model Score. Agreement across the top 25 is visibly very high (nearly all cells ≥0.9) — these are consensus extreme outliers most models independently flag, not artifacts of one dominant model.

21. Final Model / Ensemble Selection

Pipeline	Selected output	Why
v1	XGBoost + Class Weighting	Highest test PR-AUC (0.558) and ROC-AUC (0.831) of the 3 supervised candidates, with by far the fewest false positives (7 vs. 28–29) — on identical, leakage-free features, a fair comparison by construction.
research	ensemble_percentile_average	Bounded (0,1), interpretable as a per-model-agnostic empirical percentile, robust to any single unbounded model dominating the combination — explicitly preferred over weighted-average, whose raw range reaches 18.16.
research_v2	ensemble_percentile_average	Same rationale; this is the score the live dashboard actually serves. The 4 strategies agree strongly with each other in this pipeline (pairwise Spearman 0.983–1.000), so the choice of percentile-average over the alternatives has limited practical consequence for which transactions surface as high-risk.

research_v2 is the **production/deployed** pipeline (client-designated feature set, dashboard-served). research (46-feature, in-house) remains a reference/comparison build — valuable specifically because it asks "unusual for this customer" via personal-baseline/velocity features that research_v2's more population-level frequency features cannot ask, and their ρ≈0.60 agreement (§11.2) is a meaningful cross-check between two different definitions of "unusual."

22. Production Architecture

```
dashboard/  (FastAPI backend + static frontend)
  backend/api_server.py  -->  reads ONLY artifacts_research_v2/*  (no retraining, no live scoring loop)
  frontend/               -->  Overview, Transaction Explorer, Investigation Queue,
                               Model Comparison, Explainability, Account Scenario Simulator

Run: python -m uvicorn backend.api_server:app --port 8000   (from dashboard/)
```

The Investigation Queue is the only label-generating mechanism anywhere in the deployed system — any analyst decision made there is the closest thing this project has to a real, human-confirmed label, and would be the natural seed for a genuinely supervised model in a future iteration.

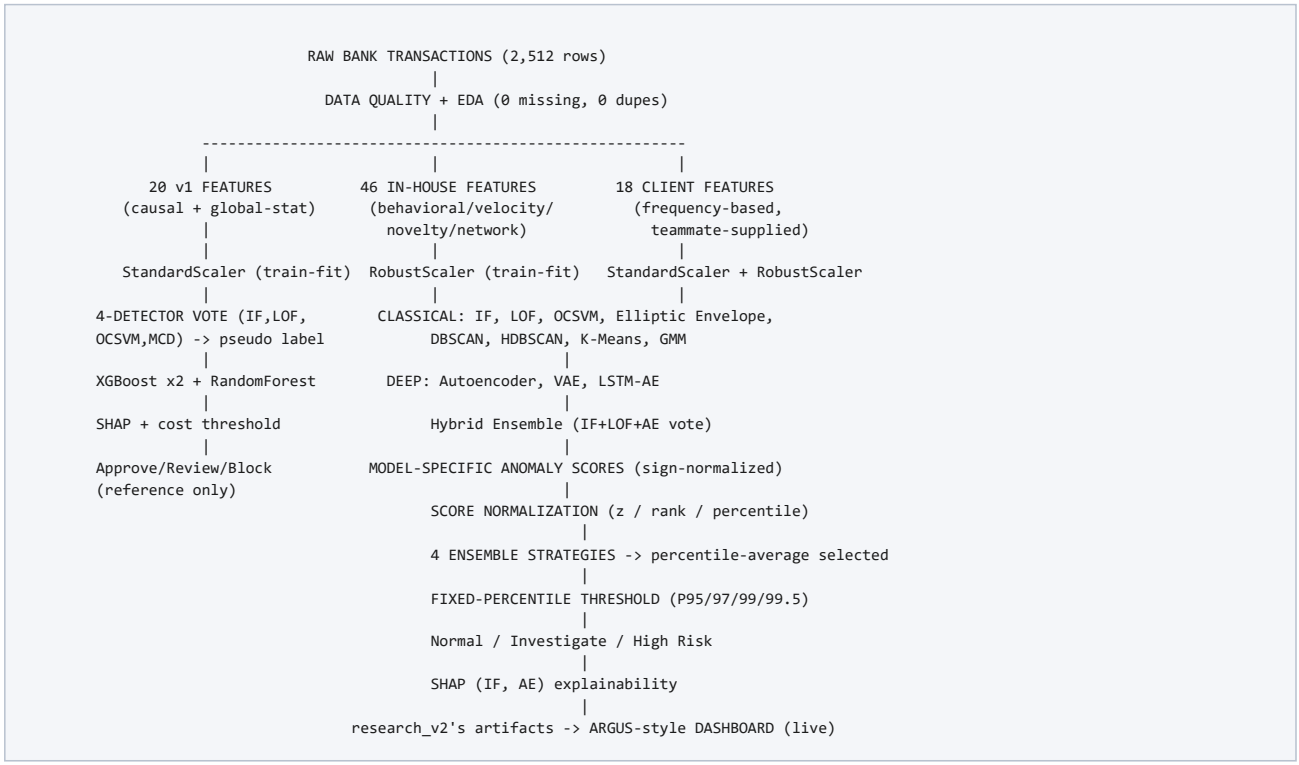
23. Drift and Monitoring

- **v1 confirmed real temporal drift** between train and val/test: Isolation Forest's anomaly rate roughly doubles out-of-fold (5.0%→9.5–12.5%), One-Class SVM's roughly quadruples (5.4%→21–23%) — meaning the 5% contamination assumption would need periodic re-fitting in a real deployment, not a one-time calibration.
- **research/research_v2 have no monitoring loop** — both are one-shot batch scoring pipelines. A production deployment would need a scheduled re-fit cadence and a drift check comparing new transactions' score distribution against the fitted training distribution.
- **DBSCAN/HDBSCAN's full-dataset fitting** means their cluster structure would need to be refit (not just re-scored) on any new data batch, unlike the other 10 models, which can score new rows against an already-fit model.

24. Limitations

- **No genuine fraud label exists anywhere in this project** — every "fraud" signal, in all three pipelines, is either an anomaly score or a pseudo-label derived from anomaly scores.
- **v1's pseudo-label is circular by construction** — generated by the same family of unsupervised detectors later explained via SHAP on the supervised model trained on that label.
- **Small dataset** — 2,512 rows total; v1's test fold has only 71 pseudo-fraud rows, and research/research_v2's val folds are 503 rows each. Every metric in this document should be read with that sample size in mind.
- **Contamination assumption (5%)** is a documented, unverified guess applied uniformly across models in both unsupervised pipelines and v1's detector voting — not derived from any ground truth.
- **Real distribution drift exists** (§23) — a static model fit once would degrade over time without re-fitting.
- **Cold-start accounts** — v1's val/test folds contain 14 and 7 accounts respectively with no prior transaction history in the training fold; handled via documented fallback logic (novelty flags default to 1), not silently dropped.
- **Threshold limitations** — v1's cost-optimal threshold (0.01) flags 90% of test rows for manual review — mathematically optimal under the stated illustrative 50:1 cost ratio, but operationally useless for a real bank. Neither unsupervised pipeline can compute a cost-optimal threshold at all, for lack of a label.
- **DBSCAN/HDBSCAN fit on the full dataset** in both 12-model pipelines (no native out-of-sample predict) — a real, bounded, and identically-present asymmetry in both pipelines, not classic test leakage, but worth fixing before any genuine held-out evaluation set is introduced.
- **Research vs. deployment gap** — this document validates internal consistency, cross-model agreement, and leakage-safety. It does not and cannot validate real-world fraud-catching performance, because no confirmed-fraud outcome data exists to validate against.

25. Complete Model-Building Diagram



26. Judge / Interview Explanation

How to explain this project in 2 minutes

"We built three complete pipelines over the same 2,512-transaction dataset, which has no real fraud labels. The first is a supervised pipeline: four unsupervised anomaly detectors vote on each transaction, their consensus becomes a pseudo-label, and we train XGBoost and Random Forest on that label — after fixing a real data-leakage bug, our honest test performance is ROC-AUC 0.83, PR-AUC 0.56. The other two pipelines are fully unsupervised: twelve different anomaly-detection models — from Isolation Forest to a variational autoencoder — run over two independently-engineered feature sets, one with 46 in-house behavioral features and one with 18 client-designated frequency features. We combine all twelve models' scores into a single ensemble using a percentile-averaging strategy, verified their agreement and disagreement patterns with correlation heatmaps, and used SHAP to explain what each model is actually reacting to. The two independent ensembles agree moderately (ρ≈0.60) despite using completely different features — that's a real cross-validation of the overall approach, not a coincidence."

What models did you train?

16 distinct configurations across three pipelines: 4 unsupervised voters + 3 supervised classifiers (XGBoost×2, Random Forest) in v1, and the same 12-model unsupervised suite (Isolation Forest, LOF, One-Class SVM, Elliptic Envelope, DBSCAN, HDBSCAN, K-Means, GMM, Autoencoder, VAE, LSTM-Autoencoder, Hybrid Ensemble) run independently over two different feature sets.

What did you train the models on?

The same 2,512 real bank transactions in every pipeline, but three different engineered feature sets — 20 (v1), 46 (in-house, behavioral/velocity/novelty-focused), and 18 (client-designated, frequency-focused) — with no fraud label anywhere in the data.

Why 12 models?

Because fraud doesn't have one mathematical shape. One transaction may be globally extreme (Isolation Forest); another only unusual next to its local peers (LOF); another may sit outside every meaningful behavioral cluster (DBSCAN/HDBSCAN); another may be an unusual combination of otherwise-normal features that only a reconstruction model notices (Autoencoder/VAE); another may only be unusual in temporal sequence (LSTM-AE). Combining diverse models reduces dependence on any single mathematical assumption about what "anomalous" means.

Why an ensemble?

Because the 12 models measurably disagree — our Jaccard/Spearman heatmaps show DBSCAN, for example, flags a largely different set of transactions than the other 11. An ensemble captures consensus at the extreme tail (our top-10 transactions are flagged by nearly every model) while still surfacing the genuine diversity lower down the ranking.

What does the heatmap show?

Several different things depending on which one: the Spearman heatmap shows how similarly two models rank all transactions; the Jaccard heatmap shows how much their specific top-5% flagged sets overlap; the feature-correlation heatmap shows which engineered features are redundant vs. complementary; the feature-by-model heatmap shows which features each model is actually reacting to.

Which model is best?

There isn't a single "best" unsupervised model — each catches a different anomaly type, which is exactly why we ensemble them rather than picking one. Isolation Forest is the most production-ready (fastest, native out-of-sample scoring). For the supervised v1 pipeline specifically, XGBoost with class weighting is the clear best of the three candidates on identical data (PR-AUC 0.558 vs. 0.468/0.431).

Why not use accuracy?

Because the classes are heavily imbalanced (~14% pseudo-fraud prevalence in v1's test fold) — a model that always predicts "normal" scores 85.9% accuracy while catching zero real fraud cases. PR-AUC specifically measures precision/recall trade-off on the minority class, which is the class that matters operationally.

Where did the fraud labels come from?

They didn't — there is no genuine fraud label anywhere in this dataset. v1's supervised label is a pseudo-label built from four unsupervised detectors' own vote consensus; it should never be described as investigator-confirmed fraud. The two unsupervised pipelines don't use labels at all.

How did you prevent data leakage?

v1 originally had real leakage — global statistics and anomaly detectors were fit on the full dataset before splitting. We fixed it with a chronological 64/16/20 train/val/test split, fitting every statistic, scaler, and detector on train only, selecting the threshold on validation only, and touching test exactly once for final numbers — which is why the honest post-fix numbers (ROC-AUC 0.83) are lower than the pre-fix numbers (≈0.94) we explicitly discarded. The two unsupervised pipelines fit 10 of 12 models train-only; DBSCAN and HDBSCAN are a documented exception (no native out-of-sample method) fit on the full dataset in both pipelines.

Is this ready for a real bank?

Not as-is, and we say so explicitly. The dataset is small (2,512 rows), there is no ground-truth fraud label to validate against, the illustrative cost figures ($5 FP / $250 FN) are placeholders not real bank numbers, and real distribution drift means any deployed model needs a re-fitting cadence. What is ready: the leakage-safety discipline, the explainability tooling (SHAP), the multi-model ensemble architecture, and the live dashboard — the pieces a real deployment would be built from, once paired with real labeled outcome data and the bank's actual cost/capacity constraints.

Appendix — Generated Artifact Index

All files below were generated fresh for this audit from the repository's own source code and artifacts.

Type	Location	Count
Heatmap PNGs (research)	`audit/heatmaps/research_*.png`	11
Heatmap PNGs (research_v2)	`audit/heatmaps/v2_*.png`	10
Backing matrix CSVs (research)	`audit/tables/research_*.csv`	11
Backing matrix CSVs (research_v2)	`audit/tables/v2_*.csv` + crosscheck JSON	11
Analysis documents	`audit/research_pipeline_analysis.md`, `audit/research_v2_pipeline_analysis.md`	2
This master report	`audit/BAF_Model_Building_Audit.pdf`	1